



**Departamentul Automatică și Informatică
Industrială**

**Facultatea Automatică și Calculatoare
Universitatea Națională de Știință și Tehnologie
POLITEHNICA București**



LUCRARE DE DIPLOMĂ

**Sistem autonom de detecție, căutare și
evitare a obstacolelor
pentru robotul patruped Unitree Go2**

Absolvent:

Radu Bălăceanu

Coordonator:

Prof.dr.ing. Ioan Ștefan Sacală

București, 2026

Cuprins

Listă de acronime	4
Lista de figuri.....	5
1 Introducere	7
2 Descrierea problemei și soluții existente	8
2.1 Descrierea problemei	8
2.2 Roboți mobili și roboți patrupezi	9
2.3 Percepția mediului în robotică mobilă.....	12
2.4 Detecția obiectelor în imagini	12
2.4.1 Metode clasice de detecție	12
2.4.2 Metode bazate pe rețele neuronale	12
2.4.3 Algoritmi de detecție în două etape și într-o singură etapă.....	13
2.4.4 Funcționarea și alegerea modelului YOLO	13
2.5 Utilizarea senzorilor LiDAR	14
2.6 Integrarea software prin ROS2, DDS și SDK	15
2.7 Soluții tehnice existente.....	16
3 Soluția propusă.....	17
3.1 Platforma experimentală și mediul software.....	17
3.2 Arhitectura generală a sistemului	18
3.3 Evoluția implementării	19
3.3.1 Procesarea externă pe laptop.....	19
3.3.2 Procesarea pe robot cu transmiterea comenzilor prin <i>subprocess</i>	20
3.3.3 Trecerea la comunicarea prin publisher ROS2	21
3.3.4 Adaptarea modulului C++ de comandă	22
3.3.5 Integrarea datelor LiDAR	24
3.3.6 Utilizarea plăcii video pentru procesarea YOLO	26
3.4 Funcționarea sistemului final.....	26
3.4.1 Achiziția imaginilor și detecția YOLO	27
3.4.2 Integrarea datelor LiDAR	27
3.4.3 Logica de decizie și prioritatea comenzilor	27
3.4.4 Transmiterea comenzilor și verificarea vizuală	28
3.5 Extensie experimentală pentru recunoașterea facială.....	28

4	Rezultate experimentale	29
4.1	Obiectivele testării	29
4.2	Configurația de test	29
4.3	Rezultate privind detecția obiectului	29
4.4	Rezultate privind evitarea obstacolelor cu LiDAR	33
4.5	Testarea sistemului integrat	39
4.6	Rezultate privind timpul de procesare	44
4.7	Test preliminar pentru recunoașterea facială	45
4.8	Limitări observate	46
5	Concluzii și dezvoltări viitoare	48
6	Listă contribuții personale	49
7	Bibliografie	50

Listă de acronime

Acronim	Semnificație
AI	Artificial Intelligence
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DDS	Data Distribution Service
GPU	Graphics Processing Unit
HOG	Histogram of Oriented Gradients
LiDAR	Light Detection and Ranging
OpenCV	Open Source Computer Vision Library
ROS2	Robot Operating System 2
RViz2	Instrument de vizualizare pentru date ROS2
SDK	Software Development Kit
SIFT	Scale-Invariant Feature Transform
SSD	Single Shot MultiBox Detector
SSH	Secure Shell
SURF	Speeded-Up Robust Features
YOLO	You Only Look Once
YOLOv8n	Varianta nano a modelului YOLOv8

Lista de figuri

Figura 2.1 Robotul patruped Unitree Go2	8
Figura 2.2 Vedere frontală a robotului Unitree Go2 (zona camerei și a senzorilor)	9
Figura 2.3 Robotul patruped Spot și principalele sale elemente constructive.	11
Figura 2.4 Utilizarea robotului ANYmal într-un mediu industrial. Sursa: [16]	11
Figura 2.5 Exemplu de detecție a obiectelor cu modelul YOLO. Sursa: [9]	13
Figura 2.6 Principiul de funcționare al unui senzor LiDAR	14
Figura 2.7 Senzorul LiDAR montat pe robotul Unitree Go2	15
Figura 3.1 Arhitectura generală a sistemului propus	18
Figura 3.2 Fluxul de funcționare al variantei cu procesare externă pe laptop	19
Figura 3.3 Transmiterea comenzii către /go2 command prin subprocess	20
Figura 3.4 Procesarea locală pe robot	20
Figura 3.5 Fragment de cod pentru publicarea unei comenzi ROS2	21
Figura 3.6 Fluxul de funcționare al variantei cu publisher ROS2	21
Figura 3.7 Fragment de cod pentru crearea publisher-ului ROS2	22
Figura 3.8 Lanțul software modulul Python - controlerul intern	23
Figura 3.9 Fragment de cod pentru abonarea modulului C++	23
Figura 3.10 Fragment de cod pentru executarea comenzii de rotire	23
Figura 3.11 Vizualizarea datelor LiDAR în RViz2	24
Figura 3.12 Exemplu de valori publicate pe topicul /utlidar/range_info	25
Figura 3.13 Exemplu de comandă de evitare generată pe baza datelor LiDAR	26
Figura 4.1 Poziționarea inițială a scaunului în partea stângă față de robot	30
Figura 4.2 Detectarea scaunului în partea stângă a imaginii și generarea comenzii de rotire spre stânga	30
Figura 4.3 Mesaje afișate în terminal pentru comanda de rotire spre stânga	31
Figura 4.4 Poziționarea robotului după rotirea către scaun	31
Figura 4.5 Detectarea scaunului în zona centrală a imaginii	32
Figura 4.6 Mesaje afișate în terminal pentru comanda de oprire	32
Figura 4.7 Detectarea scaunului în partea dreaptă a imaginii și generarea comenzii de rotire spre dreapta	33
Figura 4.8 Scenariul inițial de test, cu spațiu liber în fața robotului	34
Figura 4.9 Vizualizarea datelor LiDAR în RViz2 pentru scenariul cu direcția frontală liberă	34
Figura 4.10 Mesaje afișate în terminal pentru comanda de deplasare înainte	35
Figura 4.11 Apropierea robotului de obstacolul lateral stânga	35

<i>Figura 4.12 Vizualizarea datelor LiDAR în RViz2 pentru apropierea de obstacolul lateral</i>	<i>36</i>
<i>Figura 4.13 Mesaje afișate în terminal pentru comanda de deplasare spre dreapta .</i>	<i>36</i>
<i>Figura 4.14 Scenariu în care robotul se apropie de un obstacol frontal</i>	<i>37</i>
<i>Figura 4.15 Vizualizarea datelor LiDAR în RViz2 pentru obstacolul frontal</i>	<i>37</i>
<i>Figura 4.16 Mesaje afișate în terminal pentru comanda de rotire spre dreapta</i>	<i>38</i>
<i>Figura 4.17 Poziția robotului după rotirea de evitare</i>	<i>38</i>
<i>Figura 4.18 Vizualizarea datelor LiDAR după eliberarea direcției frontale</i>	<i>38</i>
<i>Figura 4.19 Obiectul nu este detectat, iar sistemul activează căutarea.....</i>	<i>39</i>
<i>Figura 4.20 Poziția robotului în timpul căutării.....</i>	<i>40</i>
<i>Figura 4.21 Datele LiDAR pentru deplasarea înainte</i>	<i>40</i>
<i>Figura 4.22 Rotirea spre stânga în timpul căutării</i>	<i>41</i>
<i>Figura 4.23 Poziția robotului în timpul rotirii.....</i>	<i>41</i>
<i>Figura 4.24 Rotirea spre dreapta în timpul căutării.....</i>	<i>42</i>
<i>Figura 4.25 Detectarea scaunului și corecția spre dreapta</i>	<i>42</i>
<i>Figura 4.26 Datele LiDAR în apropierea peretelui lateral</i>	<i>43</i>
<i>Figura 4.27 Poziția robotului lângă scaun</i>	<i>43</i>
<i>Figura 4.28 Detectarea scaunului și oprirea robotului</i>	<i>44</i>
<i>Figura 4.29 Exemplu de timp de procesare YOLO în varianta cu rulare pe CPU.....</i>	<i>44</i>
<i>Figura 4.30 Exemplu de timp de procesare YOLO în varianta cu rulare pe GPU.....</i>	<i>45</i>
<i>Figura 4.31 Recunoașterea persoanei și determinarea poziției acesteia față de centrul imaginii</i>	<i>45</i>
<i>Figura 4.32 Generarea comenzii de deplasare pe baza recunoașterii persoanei.....</i>	<i>46</i>

1 Introducere

Un robot mobil trebuie să poată interpreta mediul în care se află, nu doar să execute comenzile primite. Într-un spațiu real, apar obstacole, obiecte poziționate diferit, zone libere sau blocate și situații în care robotul trebuie să reacționeze rapid. Din acest motiv, percepția mediului este o componentă importantă în dezvoltarea aplicațiilor robotice.

Roboții patrupezi pot fi utilizați pentru inspecție, explorare, cercetare sau testarea unor algoritmi de percepție și control. Platforme precum Spot, ANYmal sau Unitree Go2 arată interesul actual pentru roboți capabili să combine deplasarea cu informațiile primite de la senzori.

Am ales această temă deoarece simpla comandă de mișcare a unui robot nu este suficientă într-un mediu real. În cadrul lucrării am folosit robotul patruped Unitree Go2 pentru dezvoltarea unui sistem software de detecție, căutare și evitare a obstacolelor. Scopul principal este ca robotul să poată detecta un obiect de interes, să îl caute atunci când acesta nu este vizibil, să se orienteze către el și să se apropie controlat, fără să intre în obstacole.

Pentru detecția obiectului am folosit modelul YOLOv8n. Pentru siguranța deplasării am folosit datele furnizate de senzorul LiDAR. Comenzile de mișcare sunt transmise prin ROS2 către un modul C++ care folosește SDK-ul Unitree. În acest fel, partea de percepție și decizie este separată de partea care comunică direct cu robotul.

Contribuția principală a lucrării constă în integrarea acestor componente într-un sistem funcțional testat pe robot. Implementarea a fost realizată treptat: inițial procesarea a fost făcută pe laptop, apoi a fost mutată pe unitatea de calcul a robotului, iar transmiterea comenzilor a fost îmbunătățită prin trecerea de la apeluri externe cu *subprocess* la un publisher ROS2. Ulterior, sistemul a fost completat cu date LiDAR pentru evitarea obstacolelor.

Obiectivele lucrării sunt: rularea detecției YOLO pe robot, interpretarea poziției obiectului în imagine, transmiterea comenzilor prin ROS2, adaptarea modulului de comandă bazat pe SDK-ul Unitree, integrarea datelor LiDAR și testarea sistemului în scenarii reale. Pe lângă sistemul principal, am testat și o extensie preliminară pentru recunoașterea facială, cu scopul de a verifica dacă arhitectura poate fi adaptată și pentru identificarea unei persoane cunoscute.

Lucrarea este organizată în cinci capitole. Primul capitol prezintă tema, motivația și obiectivele lucrării. Capitolul al doilea descrie problema abordată și conceptele necesare: roboți mobili, roboți patrupezi, detecția obiectelor, LiDAR, ROS2 și SDK. Capitolul al treilea prezintă soluția propusă și etapele de implementare. Capitolul al patrulea conține rezultatele experimentale obținute pe robotul Unitree Go2. Ultimul capitol prezintă concluziile și direcțiile viitoare de dezvoltare.

2 Descrierea problemei și soluții existente

În acest capitol prezint elementele tehnice necesare pentru înțelegerea sistemului dezvoltat. Am pornit de la problema urmărită în lucrare, apoi am descris pe scurt roboții mobili, roboții patrupezi, detecția obiectelor, LiDAR-ul și modul în care ROS2 și SDK-ul Unitree sunt folosite pentru integrarea componentelor software.

Capitolul include și un exemplu de soluție tehnică existentă, apropiată de tema lucrării, pentru a evidenția modul în care detecția obiectelor poate fi folosită pe o platformă robotică patrupeadă.

2.1 Descrierea problemei

În această lucrare am dezvoltat un sistem software pentru robotul patrupeț Unitree Go2. Scopul este ca robotul să poată detecta un obiect, să se orienteze către acesta și să evite obstacolele din apropiere.



Figura 2.1 Robotul patrupeț Unitree Go2

Problema abordată nu se limitează la detectarea unui obiect într-o imagine, ci presupune integrarea acestei funcționalități într-un sistem robotic capabil să ia decizii simple de deplasare.

Camera video conectată la robot reprezintă principala sursă de informație vizuală, iar zona frontală în care sunt amplasați senzorii este prezentată în figura 2.2.

Pentru fiecare cadru video, sistemul verifică dacă obiectul urmărit este prezent în imagine. Dacă acesta este detectat, robotul se orientează în funcție de poziția obiectului: se rotește spre stânga, spre dreapta sau avansează dacă obiectul este centrat.

Înainte de deplasare, robotul verifică și existența obstacolelor folosind datele furnizate de LiDAR, pentru a se asigura că direcția de mers este liberă.

Deoarece robotul funcționează într-un mediu real, sistemul trebuie să țină cont atât de timpul de procesare, cât și de siguranța mișcării.

Scopul lucrării este realizarea unui sistem care combină achiziția imaginilor, detecția obiectului, evitarea obstacolelor și generarea comenzilor de deplasare pentru robot.



Figura 2.2 Vedere frontală a robotului Unitree Go2 (zona camerei și a senzorilor)

2.2 Roboți mobili și roboți patrupezi

Roboții mobili sunt sisteme robotice care se pot deplasa autonom sau semiautonom și pot interacționa cu mediul înconjurător. În funcție de modul de deplasare, aceștia pot fi roboți pe roți, roboți pe șenile, roboți zburători sau roboți cu picioare [2].

Roboții cu picioare sunt importanți deoarece se pot adapta mai bine la terenuri neregulate și pot depăși obstacole care ar fi dificile pentru alte tipuri de platforme. O categorie importantă este cea a roboților patrupezi, inspirați de modul de deplasare al

animalelor. Acești roboți sunt folosiți în aplicații precum inspecția industrială, explorarea zonelor periculoase și cercetarea în robotică.

Un exemplu cunoscut este robotul Spot, dezvoltat de Boston Dynamics. Spot este o platformă patrupedă folosită pentru inspecție, colectare de date și deplasare în medii dificile sau periculoase pentru oameni [15]. Figura 2.3 prezintă robotul Spot și principalele sale elemente constructive. Un astfel de robot poate fi echipat cu senzori și module suplimentare, în funcție de aplicația urmărită.

Din punct de vedere al percepției, Spot folosește camere amplasate pe corpul robotului pentru a obține informații despre mediul apropiat. Aceste date sunt utilizate pentru estimarea obstacolelor și pentru deplasarea în siguranță. Astfel, robotul nu se bazează doar pe comenzi simple de mișcare, ci pe un sistem intern capabil să gestioneze stabilitatea și percepția mediului.

Din punct de vedere software, Spot poate fi comandat și printr-un SDK care oferă interfețe de nivel înalt [15]. Utilizatorul nu comandă direct fiecare motor sau fiecare articulație, ci trimite comenzi prin servicii software. Controlul locomotor de bază rămâne gestionat de sistemul intern al robotului, iar aplicațiile dezvoltate extern implementează logica de nivel superior.

Un alt exemplu relevant este ANYmal, dezvoltat de ANYbotics. Acest robot este orientat mai ales către inspecția industrială autonomă și este proiectat pentru medii dificile [16]. Figura 2.4 arată utilizarea unei astfel de platforme într-un mediu industrial. Față de alte platforme, ANYmal este conceput mai robust, astfel încât să poată funcționa în condiții mai dure.

Conform specificațiilor producătorului, ANYmal are protecție IP67, ceea ce înseamnă că este protejat împotriva prafului și apei [17]. Din acest motiv, poate fi utilizat în medii umede, prăfuite sau în zone unde prezența umană poate fi riscantă. Acest exemplu arată direcția în care evoluează roboții patrupezi industriali: platforme robuste, capabile să colecteze date și să execute misiuni în medii reale.

Exemple precum Spot și ANYmal arată că roboții patrupezi moderni nu sunt doar platforme de deplasare, ci sisteme complexe care combină senzori, algoritmi de percepție, software de comandă și control intern al locomoției. În aceste sisteme, utilizatorul lucrează de obicei cu interfețe software de nivel înalt, nu cu control direct asupra fiecărui actuator.

În comparație cu aceste platforme comerciale avansate, în lucrarea de față folosesc robotul Unitree Go2 ca platformă experimentală. Scopul meu este dezvoltarea unei soluții autonome, bazate pe detecție vizuală, date LiDAR și comenzi transmise prin SDK-ul Unitree [13], [14]. Mai exact, urmăresc obținerea unui comportament simplu de căutare a obiectului, orientare către acesta și evitare a obstacolelor apropiate.

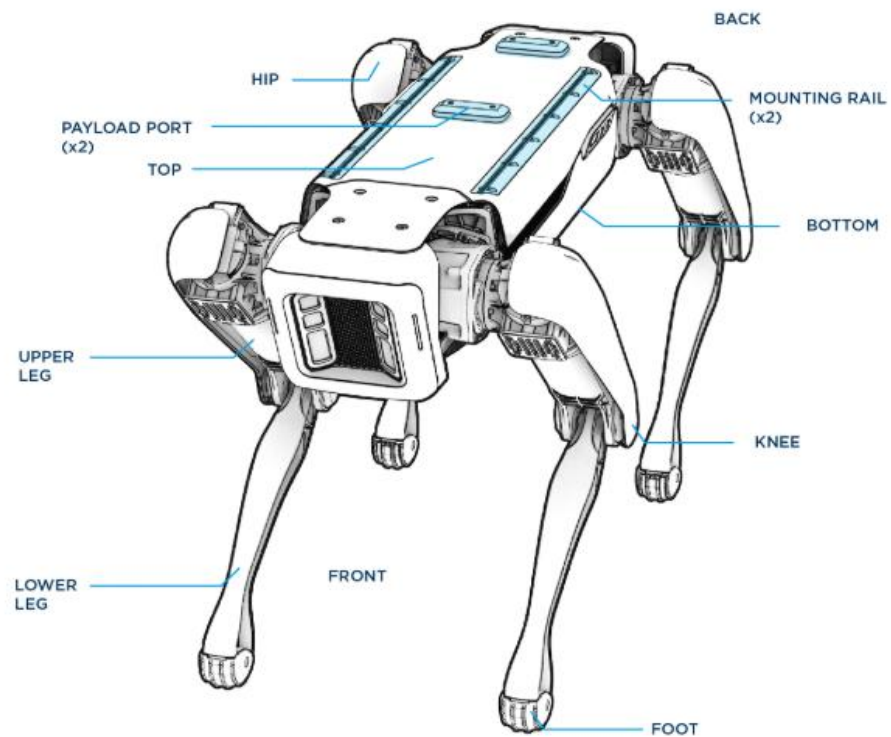


Figura 2.3 Robotul patruped Spot și principalele sale elemente constructive. Sursa: [15].



Figura 2.4 Utilizarea robotului ANYmal într-un mediu industrial. Sursa: [16]

2.3 Percepția mediului în robotică mobilă

Pentru a se deplasa într-un mediu real, un robot mobil trebuie să primească informații despre spațiul din jur. Pe baza acestor informații, robotul poate identifica obiecte, poate evita obstacole și poate lua decizii de mișcare [2].

Percepția mediului se realizează cu ajutorul senzorilor. Camerele video oferă informații vizuale și sunt utile pentru identificarea obiectelor din imagine. Totuși, rezultatele obținute cu o cameră pot fi influențate de lumină, perspectivă, mișcarea robotului sau calitatea imaginii [1].

Senzorii LiDAR oferă informații despre distanța până la obstacole. Aceste date sunt importante pentru deplasarea în siguranță, deoarece robotul poate verifica dacă direcția în care urmează să se miște este liberă [2].

Cele două tipuri de informații au roluri diferite. Camera poate ajuta robotul să recunoască obiectul urmărit, iar LiDAR-ul poate verifica dacă deplasarea către acel obiect se poate face fără risc de coliziune.

În această lucrare am folosit cele două surse de informație împreună. Camera și modelul YOLO sunt folosite pentru detectarea obiectului urmărit [3], iar LiDAR-ul este folosit pentru evitarea obstacolelor și adaptarea mișcării robotului.

2.4 Detectția obiectelor în imagini

Detectția obiectelor în imagini reprezintă procesul prin care un sistem identifică obiectele prezente într-o imagine și stabilește poziția lor. Rezultatul unei detecții este format, de obicei, din clasa obiectului, scorul de încredere și caseta de încadrare. În robotică, această etapă este importantă deoarece îi permite robotului să interpreteze scena și să ia decizii pe baza informațiilor vizuale [1].

2.4.1 Metode clasice de detecție

Primele metode de detecție s-au bazat pe reguli definite manual și pe extragerea unor caracteristici din imagine. Din această categorie fac parte segmentarea după culoare, detecția conturilor și descriptorii vizuali precum HOG, SIFT sau SURF [1].

Aceste metode pot funcționa bine în situații simple și controlate. Totuși, ele devin mai puțin eficiente atunci când apar variații de iluminare, schimbări de perspectivă sau fundaluri complexe. Din acest motiv, pentru lucrarea de față am ales o metodă bazată pe rețele neuronale, mai potrivită pentru detecția obiectelor în medii reale.

2.4.2 Metode bazate pe rețele neuronale

În prezent, detecția obiectelor se realizează frecvent cu ajutorul rețelelor neuronale convoluționale. Aceste modele învață automat caracteristicile importante dintr-un număr mare de imagini și pot oferi rezultate mai bune în scene variate.

Un astfel de model trebuie să rezolve două probleme în același timp. Prima este clasificarea obiectului, adică stabilirea tipului de obiect detectat. A doua este localizarea lui în imagine, de obicei printr-o casetă de încadrare.

2.4.3 Algoritmi de detecție în două etape și într-o singură etapă

Algoritmii moderni de detecție se împart, în general, în două categorii. Prima categorie este cea a algoritmilor în două etape, cum sunt R-CNN, Fast R-CNN și Faster R-CNN [4], [5], [6]. Aceștia caută mai întâi zonele de interes din imagine și apoi clasifică obiectele din acele zone. În general, oferă o acuratețe ridicată, dar sunt mai lenți.

A doua categorie este cea a algoritmilor într-o singură etapă, precum SSD și YOLO [7], [3]. Aceștia realizează detecția într-un singur pas, ceea ce îi face mai rapizi și mai potriviți pentru aplicații în timp real.

În cazul roboților mobili, viteza de procesare este foarte importantă. Robotul trebuie să reacționeze rapid la schimbările din mediu, astfel că un model prea lent poate afecta comportamentul sistemului.

2.4.4 Funcționarea și alegerea modelului YOLO

În această lucrare am ales modelul YOLO, prescurtare de la *You Only Look Once*, deoarece este potrivit pentru detecția obiectelor în timp real. Acesta este un algoritm de detecție într-o singură etapă, ceea ce înseamnă că analizează imaginea o singură dată și returnează direct obiectele detectate, scorurile de încredere și casetele de încadrare [3].

Principiul de funcționare al modelului constă în împărțirea imaginii în zone și estimarea directă a obiectelor care pot apărea în acestea. Pentru fiecare posibilă detecție, modelul estimează clasa obiectului, poziția casetei de încadrare și un scor de încredere. Detecțiile cu scor redus sunt eliminate, iar dintre casetele care se suprapun este păstrată cea mai relevantă.

Spre deosebire de metodele în două etape, acest tip de algoritm nu caută separat regiuni de interes care să fie clasificate ulterior. Poziția și clasa obiectelor sunt estimate direct, ceea ce reduce timpul de procesare și face metoda utilă pentru aplicații robotice.

În cadrul lucrării, modelul este folosit pentru detectarea obiectului urmărit în cadrul video curent. Poziția casetei de încadrare permite sistemului să stabilească dacă obiectul se află în stânga, în dreapta sau în zona centrală a imaginii.

Un exemplu de rezultat al detecției YOLO este prezentat în figura 2.5.



Figura 2.5 Exemplu de detecție a obiectelor cu modelul YOLO. Sursa: [9]

În această lucrare, modelul YOLO nu este antrenat de la zero, ci este integrat într-un sistem robotic care transformă rezultatul detecției în comenzi simple de mișcare.

2.5 Utilizarea senzorilor LiDAR

LiDAR-ul, denumit *Light Detection and Ranging*, este un senzor folosit pentru măsurarea distanței până la obiectele din jur. Principiul său de funcționare se bazează pe emiterea unui impuls laser și pe măsurarea timpului necesar pentru ca semnalul reflectat de un obstacol să se întoarcă la senzor.

În figura 2.6 este prezentat principiul general de funcționare al unui senzor LiDAR. Deoarece semnalul parcurge drumul dus-întors, distanța până la obstacol se calculează folosind jumătate din distanța totală parcursă de semnal. În robotică mobilă, este util pentru detectarea obstacolelor și pentru identificarea zonelor libere de deplasare. Dacă un obstacol este detectat prea aproape de robot, sistemul poate opri deplasarea înainte sau poate genera o comandă de evitare.

În figura 2.7 este prezentat senzorul LiDAR utilizat pe robotul Unitree Go2. În cadrul acestei lucrări, LiDAR-ul are rol de mecanism de siguranță. Datele furnizate de senzor sunt folosite pentru a verifica dacă direcția de mers este liberă. Atunci când distanța față de un obstacol scade sub un prag stabilit, comanda de avansare este blocată și înlocuită cu o comandă de oprire sau de evitare.

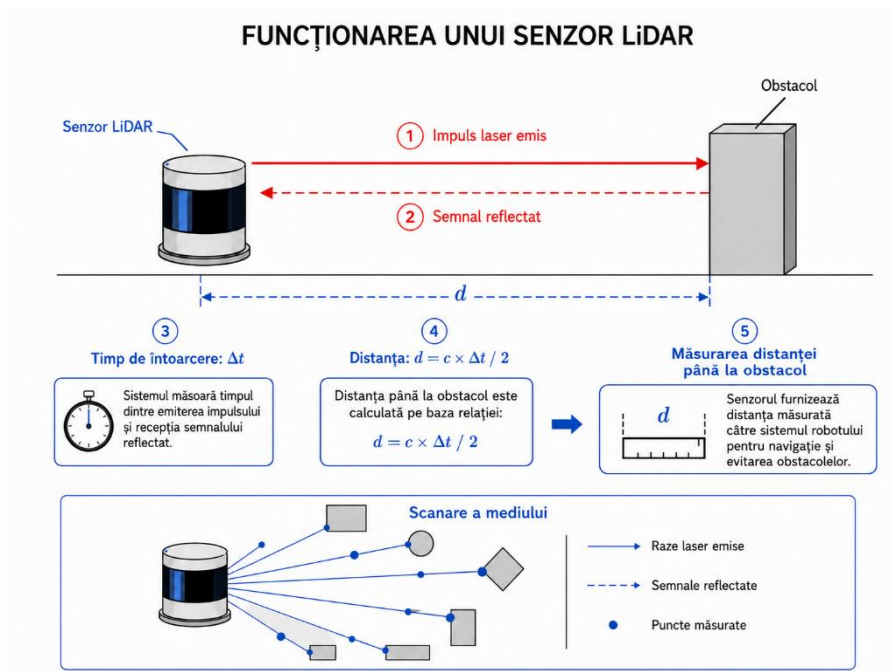


Figura 2.6 Principiul de funcționare al unui senzor LiDAR



Figura 2.7 Senzorul LiDAR montat pe robotul Unitree Go2

2.6 Integrarea software prin ROS2, DDS și SDK

În implementarea realizată, modularitatea a fost necesară pentru separarea logicii de percepție și decizie de partea care comunică direct cu robotul. Modulul Python se ocupă de detecția YOLO, citirea datelor LiDAR, stabilirea comenzii finale și publicarea acesteia pe topicul ROS2 `/go2_command`. Modulul C++ preia comanda publicată pe acest topic și o transformă într-o mișcare executată de robot prin SDK-ul Unitree.

Pentru comunicarea dintre aceste componente am folosit ROS2. Acesta organizează aplicațiile în noduri, topicuri și mesaje. Un nod poate publica informații pe un topic, iar alt nod se poate abona la acel topic pentru a primi datele transmise [11].

La baza comunicației din ROS2 se află DDS, prescurtare de la Data Distribution Service. DDS permite schimbul de date între procese diferite printr-un model de tip *publish/subscribe*. Acest mod de lucru este util în robotică, deoarece permite separarea componentelor software și comunicarea eficientă între ele [12].

Pe lângă ROS2, roboții folosesc de obicei și SDK-uri oferite de producători. Un SDK pune la dispoziție funcții prin care aplicațiile externe pot comunica cu robotul. Astfel, dezvoltatorul nu controlează direct motoarele sau articulațiile, ci trimite comenzi de nivel înalt către sistemul intern al platformei.

Această separare este importantă. Robotul își păstrează controlul locomotor de bază în sistemul intern, iar aplicațiile externe se ocupă de prelucrarea datelor, luarea deciziilor și transmiterea comenzilor. În acest fel, fiecare componentă are un rol clar, iar sistemul poate fi modificat mai ușor.

În cazul robotului Unitree Go2, SDK-ul este folosit pentru transmiterea comenzilor către robot și pentru comunicarea cu sistemul intern al platformei [13], [14]. În lucrarea de față, comenzile nu sunt trimise direct către motoare, ci sunt transmise printr-un modul software care folosește interfața oferită de Unitree.

2.7 Soluții tehnice existente

Un exemplu relevant de soluție tehnică existentă este aplicația Spot Tensorflow Object Detection din documentația oficială Boston Dynamics. Aceasta folosește imaginile captate de camerele robotului Spot și aplică un model TensorFlow pentru detecția obiectelor [18]. Programul permite detectarea unor clase de obiecte și afișarea rezultatelor prin casete de încadrare.

Acest exemplu este apropiat de lucrarea de față deoarece arată cum un robot patruped poate folosi detecția obiectelor pentru a interpreta mediul. În ambele cazuri, camera este folosită pentru identificarea obiectelor din jur, iar rezultatul detecției poate fi utilizat mai departe în logica robotului.

3 Soluția propusă

Soluția propusă constă într-un sistem software implementat pe robotul patruped Unitree Go2, cu scopul de a permite detectarea unui obiect de interes, căutarea acestuia atunci când nu este vizibil și evitarea obstacolelor din apropiere. Sistemul urmărește realizarea unei navigații autonome.

Pentru detecția obiectului am folosit modelul YOLO, aplicat pe imaginile preluate de la camera video conectată la robot. Poziția obiectului în imagine este folosită pentru orientarea robotului, iar datele furnizate de senzorul LiDAR sunt utilizate pentru verificarea obstacolelor din față și din zonele laterale. Comenzile de mișcare sunt publicate prin ROS2 pe topicul `/go2_command`, apoi sunt preluate de modulul C++ și transformate în mișcări executate prin SDK-ul Unitree.

În varianta finală, comportamentul robotului este împărțit în trei cazuri principale. Primul caz este urmărirea obiectului detectat. Atunci când obiectul apare în imagine, sistemul îi calculează poziția față de centrul cadrului și stabilește dacă robotul trebuie să se rotească spre stânga, spre dreapta, să avanseze sau să se oprească.

Al doilea caz este căutarea obiectului. Acest comportament este activat atunci când YOLO nu detectează obiectul în cadrul curent. În această situație, robotul execută mișcări simple de explorare, prin deplasări scurte și rotații, pentru a aduce obiectul în câmpul vizual al camerei.

Al treilea caz este evitarea obstacolelor. Dacă datele LiDAR indică un obstacol apropiat, sistemul blochează comanda de avansare și generează o comandă de oprire, rotire sau corecție laterală. Astfel, siguranța deplasării are prioritate față de urmărirea obiectului.

Cele trei comportamente sunt integrate într-o singură buclă de decizie. La fiecare ciclu, aplicația citește imaginea de la camera video conectată la robot, rulează detecția YOLO, verifică valorile LiDAR și stabilește comanda finală.

3.1 Platforma experimentală și mediul software

Am realizat implementarea pe robotul patruped Unitree Go2. Procesarea imaginilor, preluarea datelor LiDAR și generarea deciziilor de mișcare au fost realizate pe unitatea de calcul integrată a robotului. Astfel, sistemul nu depinde de trimiterea continuă a imaginilor către un calculator extern, iar decizia de mișcare este generată local, direct pe platforma robotică.

În această configurație, robotul rulează modulele software necesare pentru percepție, decizie și comandă, iar laptopul are rol de stație de lucru pentru dezvoltare, conectare și monitorizare. Sistemul de operare folosit pe laptop a fost Ubuntu 22.04, prin care am accesat robotul prin SSH, am editat codul, am lansat programele de test și am urmărit topicurile ROS2.

Mediul software folosit include Python, OpenCV, YOLO, ROS2 și C++. Am folosit Python pentru modulul de percepție și decizie, OpenCV pentru achiziția imaginilor de la cameră [8], [10] și YOLO pentru detecția obiectului. Comunicarea dintre module a fost realizată prin ROS2, iar partea care transformă comenzile în mișcări ale robotului a fost implementată în C++.

Pentru comunicarea cu sistemul intern al robotului am folosit SDK-ul Unitree. Acesta permite transmiterea unor comenzi de nivel înalt către robot. Sistemul dezvoltat transmite comenzi precum deplasare înainte, rotire, deplasare laterală sau oprire, iar execuția efectivă a mișcării rămâne gestionată de interfața software a robotului.

3.2 Arhitectura generală a sistemului

Am organizat arhitectura sistemului în două părți principale. Prima parte este modulul de percepție și decizie, implementat în Python. A doua parte este modulul de comandă al robotului, implementat în C++.

Modulul Python citește imaginile de la camera video conectată la robot, rulează detecția YOLO și preia datele LiDAR. Pe baza acestor informații, sistemul stabilește comanda finală pentru robot. Poziția obiectului detectat este folosită pentru orientarea robotului, iar datele LiDAR sunt folosite pentru verificarea siguranței deplasării.

După stabilirea deciziei, modulul Python publică o comandă simbolică pe un topic ROS2 dedicat comenzilor de deplasare. Această comandă nu acționează direct robotul. Ea este preluată de modulul C++ de comandă, care o interpretează și o transformă într-un apel compatibil cu SDK-ul Unitree.

Fluxul general al sistemului este reprezentat în figura 3.1.

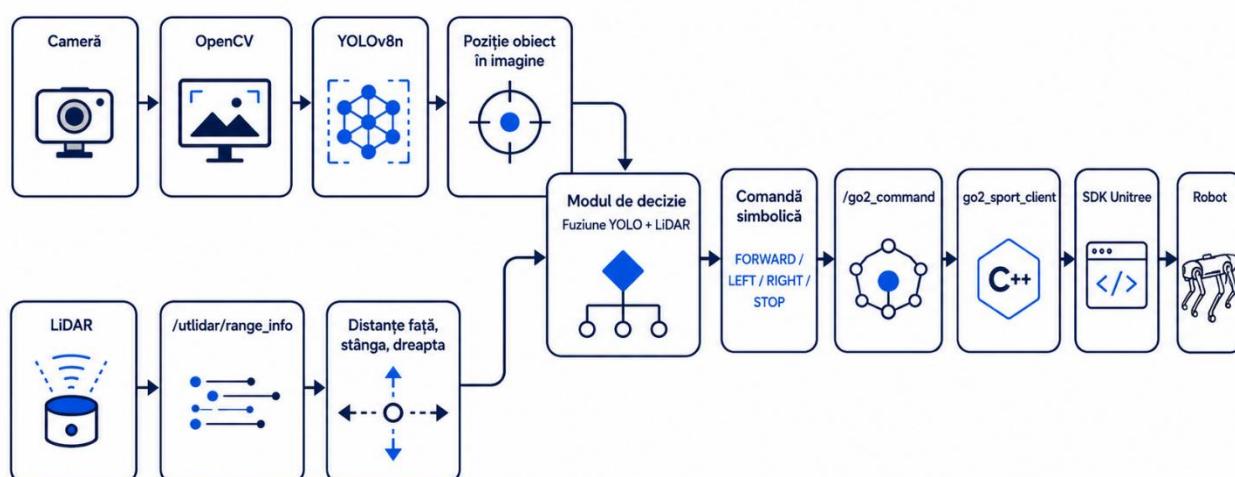


Figura 3.1 Arhitectura generală a sistemului propus

În implementarea mea, topicul ROS2 folosit pentru transmiterea comenzilor este /go2_command, iar componenta C++ care preia aceste comenzi este programul go2_sport_client. Aceste denumiri sunt specifice implementării, însă rolul lor este general: topicul transmite comanda, iar modulul C++ o transformă într-o mișcare executabilă de robot.

Această separare m-a ajutat să modific mai ușor logica de detecție și decizie, fără să rescriu componenta care comunică direct cu robotul. În același timp, controlul locomotor de bază rămâne gestionat de sistemul intern al platformei Unitree.

3.3 Evoluția implementării

Am dezvoltat sistemul treptat, prin mai multe variante succesive. După fiecare variantă am făcut teste practice și am observat ce funcționa corect, ce limita sistemul și ce trebuia modificat în etapa următoare.

3.3.1 Procesarea externă pe laptop

Prima variantă a sistemului a folosit laptopul ca unitate principală de procesare. În această etapă, robotul captura imaginile și executa mișcările, iar laptopul realiza detecția obiectului și lua decizia de orientare.

Fluxul de funcționare al acestei variante este prezentat în figura 3.2.

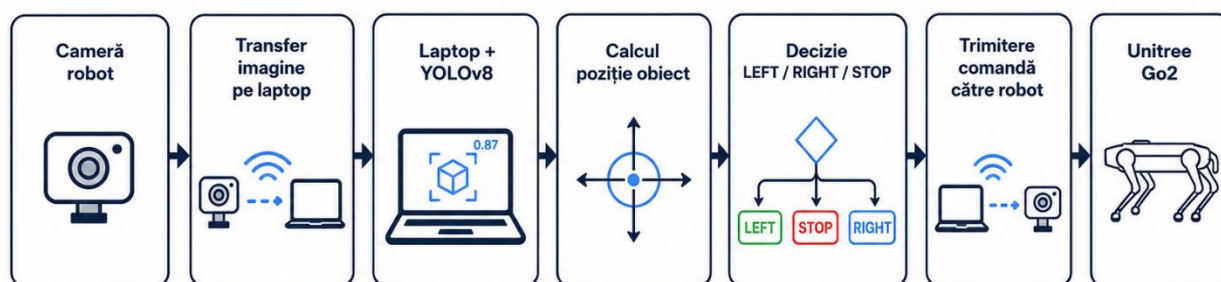


Figura 3.2 Fluxul de funcționare al variantei cu procesare externă pe laptop

În această variantă, robotul salva cadrul video sub forma unei imagini, iar laptopul o prelua și rula modelul YOLO pentru detectarea obiectului. Pe baza poziției obiectului față de centrul imaginii, laptopul genera comenzi simple de orientare: rotire la stânga, rotire la dreapta sau oprire.

Scopul acestei etape a fost verificarea funcționalității de bază: detectarea obiectului, calcularea poziției sale și orientarea robotului către țintă.

Comenzile erau transmise prin ROS2 către modulul C++ de pe robot, care le transforma în comenzi de mișcare folosind SDK-ul Unitree. Transferul imaginilor și transmiterea comenzilor erau realizate prin apeluri externe din Python, folosind *subprocess*. Un exemplu de transmitere a unei comenzi către robot este prezentat în figura 3.3.

Principala limitare a fost latența, deoarece imaginea trebuia transferată pe laptop, iar comanda trimisă apoi înapoi către robot. Din acest motiv, această variantă a fost folosită doar pentru testarea inițială, următorul pas fiind mutarea procesării direct pe robot.

```

remote_cmd = (
    "source ~/unitree_ros2/setup.sh && "
    "cd ~/unitree_ros2/example && "
    f"ros2 topic pub /go2_command std_msgs/msg/String "
    f"'{{data: \"{{command_name}}\"}}' --once"
)

try:
    result = subprocess.run(
        ["ssh", ROBOT_HOST, remote_cmd],
        capture_output=True,
        text=True,
        timeout=4
    )

```

Figura 3.3 Transmiterea comenzii către /go2 command prin subprocess

3.3.2 Procesarea pe robot cu transmiterea comenzilor prin *subprocess*

După testarea variantei cu procesare pe laptop, am mutat detecția YOLO pe unitatea de calcul a robotului. În această etapă, imaginea nu mai era transferată către laptop, ci era citită și analizată direct pe robot, folosind OpenCV.

Fluxul de funcționare al acestei variante este prezentat în figura 3.4.

Modelul YOLO rula local pe robot și detecta obiectul urmărit. Sistemul calcula poziția obiectului față de centrul imaginii și genera o comandă de orientare: rotire spre stânga, rotire spre dreapta sau oprire.

Totuși, transmiterea comenzilor era realizată în continuare prin *subprocess*. Pentru fiecare comandă era pornit un proces extern care publica un mesaj ROS2 pe topicul de comandă. Un exemplu al acestei metode este prezentat în figura 3.5. Mesajul publicat era apoi preluat de modulul C++ și transformat într-o mișcare prin SDK-ul Unitree.

Această variantă a redus latența produsă de transferul imaginilor prin rețea, dar transmiterea comenzilor a rămas inefficientă. Din acest motiv, în etapa următoare am înlocuit mecanismul bazat pe *subprocess* cu un publisher ROS2 integrat direct în nodul Python.

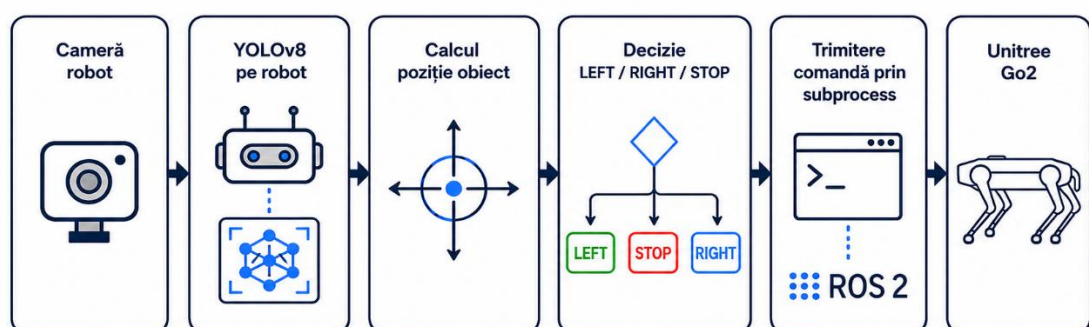


Figura 3.4 Procesarea locală pe robot

```

remote_cmd = (
    "source ~/unitree_ros2/setup.sh && "
    "cd ~/unitree_ros2/example && "
    f"ros2 topic pub /go2_command std_msgs/msg/String "
    f"'{{{data: \"{command_name}\"}}}' --once"
)

try:
    result = subprocess.run(
        ["ssh", ROBOT_HOST, remote_cmd],
        capture_output=True,
        text=True,
        timeout=4
    )

```

Figura 3.5 Fragment de cod pentru publicarea unei comenzi ROS2

3.3.3 Trecerea la comunicarea prin publisher ROS2

După varianta cu *subprocess*, am modificat modul în care erau transmise comenzile către robot. În loc să pornesc un proces extern pentru fiecare comandă, am creat un publisher ROS2 direct în programul Python. În această variantă, publisher-ul este inițializat o singură dată la pornirea programului, iar comenzile sunt publicate direct pe topicul de comandă.

Fluxul de funcționare al acestei variante este prezentat în figura 3.6.

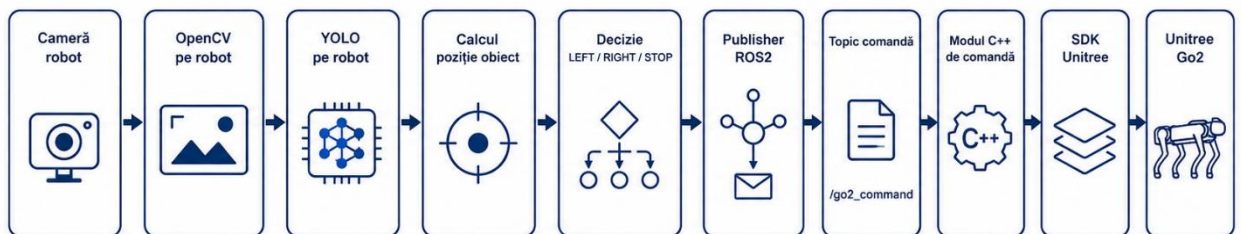


Figura 3.6 Fluxul de funcționare al variantei cu publisher ROS2

Această modificare a făcut comunicarea mai directă. Programul Python nu mai execută o comandă de shell pentru fiecare mesaj transmis, ci publică direct comanda pe topicul `/go2_command`. Mesajele publicate sunt de tip `std_msgs/String` și reprezintă comenzi simbolice, precum rotire la stânga, rotire la dreapta sau oprire.

Un fragment din codul folosit pentru crearea publisher-ului ROS2 în modulul Python este prezentat în figura 3.7.

```

class Go2CommandPublisher(Node):
    def __init__(self):
        super().__init__("go2_command_publisher")
        self.publisher_ = self.create_publisher(
            String,
            "/go2_command",
            10
        )

    def send(self, command_name: str):
        msg = String()
        msg.data = command_name
        self.publisher_.publish(msg)

```

Figura 3.7 Fragment de cod pentru crearea publisher-ului ROS2

Comenzile publicate pe `/go2_command` sunt preluate de programul `go2_sport_client`. Acesta interpretează mesajele primite și le transformă în apeluri către SDK-ul Unitree. Astfel, modulul Python stabilește comanda, iar componenta C++ o traduce în mișcări ale robotului prin funcțiile oferite de SDK.

Această etapă a fost o îmbunătățire față de varianta bazată pe *subprocess*, deoarece a redus întârzierile și a făcut comunicarea dintre modulul Python și modulul C++ mai stabilă. În această etapă, comportamentul robotului era încă limitat la orientare și oprire. Deplasarea înainte, evitarea obstacolelor și comportamentul de căutare au fost adăugate în etapele următoare.

3.3.4 Adaptarea modulului C++ de comandă

După introducerea publisher-ului ROS2 în modulul Python, a fost necesar să adaptez mult mai mult partea care traduce comenzile primite în mișcări ale robotului.

Pentru această etapă am modificat codul programului `go2_sport_client`, scris în C++. Acesta are rolul de legătură între modulul Python de decizie și sistemul intern de control al robotului. Practic, Python stabilește ce comandă trebuie executată, iar modulul C++ o traduce în mișcări ale robotului prin funcțiile oferite de SDK.

Lanțul software folosit pentru transmiterea comenzilor este prezentat în figura 3.8.

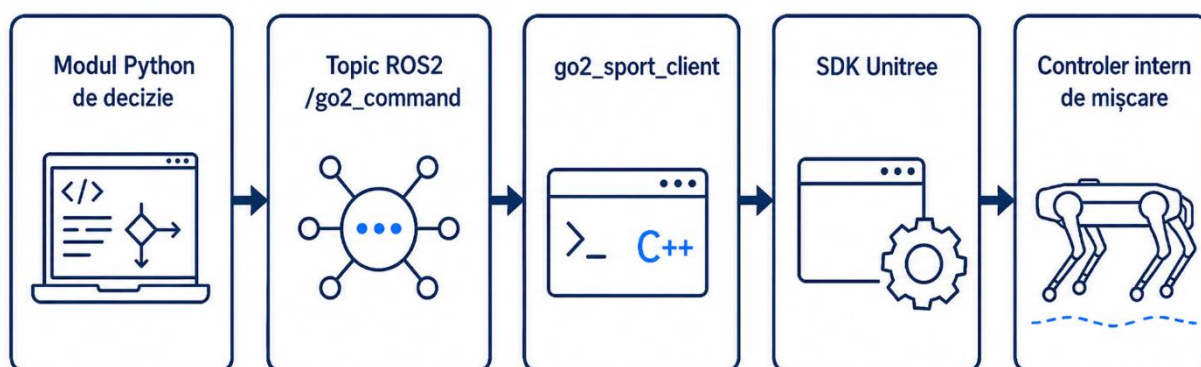


Figura 3.8 Lanțul software modulul Python - controlerul intern

În implementare, modulul C++ creează un subscriber ROS2 pentru topicul /go2_command. Atunci când primește un mesaj, conținutul acestuia este salvat ca o comandă curentă. Această comandă este apoi folosită pentru alegerea mișcării care trebuie executată de robot. Fragmentul de cod pentru abonarea la topicul de comandă /go2_command este prezentat în figura 3.9.

```

command_sub_ = this->create_subscription<std_msgs::msg::String>(
    "/go2_command", 10,
    [this](const std_msgs::msg::String::SharedPtr msg) {
        current_command_ = msg->data;
    });
  
```

Figura 3.9 Fragment de cod pentru abonarea modulului C++

Executarea comenzii se face prin funcția de control a modulului C++. Programul verifică valoarea comenzii curente și apelează funcțiile corespunzătoare din SDK-ul Unitree. Pentru mișcare este folosită funcția Move(), iar pentru oprire este folosită funcția StopMove(). Un exemplu pentru comanda de rotire spre stânga este prezentat în figura 3.10.

```

void RobotControl() {
    if (current_command_ == "ROTATE_LEFT") {
        sport_client_.Move(req_, 0.0, 0.0, 0.4);
        std::this_thread::sleep_for(std::chrono::milliseconds(800));
        sport_client_.StopMove(req_);
        current_command_ = "STOP";
    }
}
  
```

Figura 3.10 Fragment de cod pentru executarea comenzii de rotire

Această separare este importantă deoarece păstrează partea de percepție și decizie în Python, unde sunt folosite OpenCV, YOLO și ROS2. În același timp, partea de comandă rămâne în C++, unde este folosit SDK-ul Unitree. Astfel, pot modifica

logica de detecție sau decizie fără să rescriu componenta care comunică direct cu robotul.

3.3.5 Integrarea datelor LiDAR

După ce am stabilizat partea de detecție vizuală și transmiterea comenzilor prin ROS2, următorul pas a fost integrarea senzorului LiDAR. Detecția cu YOLO îmi permitea să identific obiectul urmărit, dar nu era suficientă pentru deplasarea în siguranță. Robotul putea detecta obiectul și putea genera comanda de avansare, însă între robot și țintă putea exista un obstacol.

Din acest motiv, am avut nevoie de un mecanism separat pentru detectarea obstacolelor. În prima etapă, am verificat datele LiDAR în RViz2, pentru a înțelege cum este reprezentat mediul din jurul robotului. Vizualizarea datelor LiDAR în RViz2 este prezentată în figura 3.11.

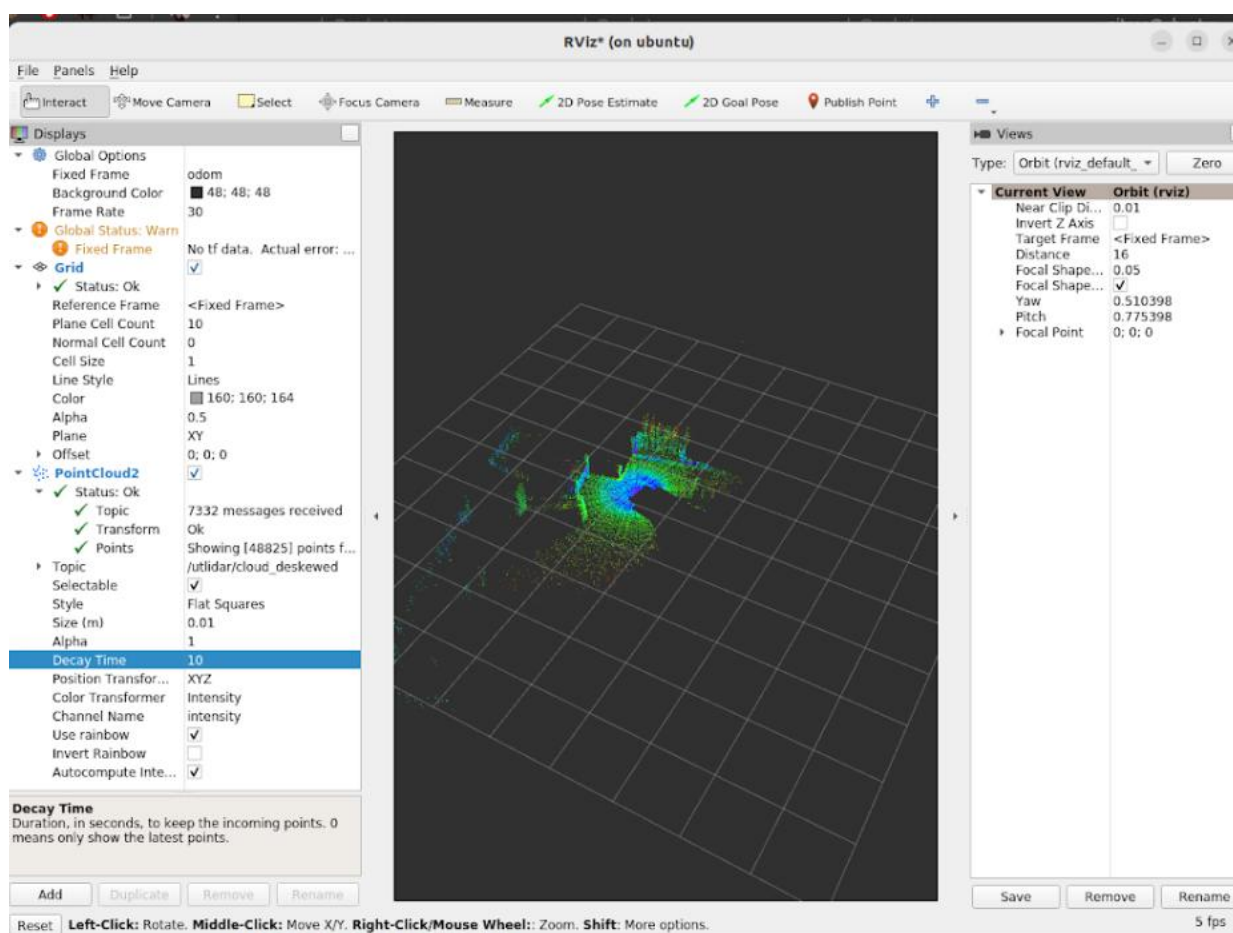
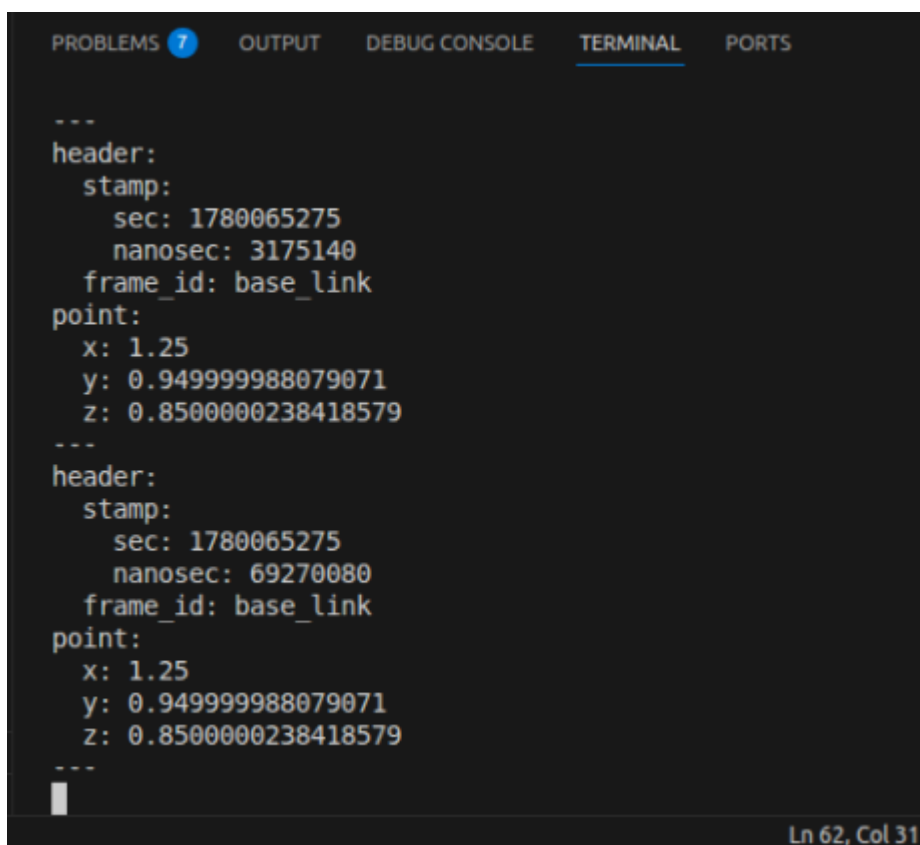


Figura 3.11 Vizualizarea datelor LiDAR în RViz2

Inițial, am analizat topicurile /utlidar/cloud și /utlidar/cloud_deskewed, care publică norul de puncte al senzorului. Aceste date au fost utile pentru a observa poziția punctelor detectate și pentru a înțelege cum descrie LiDAR-ul obstacolele din jurul robotului.

Totuși, utilizarea directă a norului de puncte s-a dovedit mai greu de folosit în practică. A fost necesară filtrarea punctelor, alegerea unor zone de interes și eliminarea valorilor instabile sau nerelevante. Din acest motiv, am folosit această variantă doar pentru testare și pentru înțelegerea funcționării senzorului.

După aceste teste, am ales să folosesc topicul `/utlidar/range_info`, deoarece acesta oferă direct distanțe relevante pentru deplasare. Din acest topic am preluat valorile pentru direcțiile față, stânga și dreapta. Un exemplu de valori publicate pe acest topic este prezentat în figura 3.12.



```
PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS

---
header:
  stamp:
    sec: 1780065275
    nanosec: 3175140
  frame_id: base_link
point:
  x: 1.25
  y: 0.949999988079071
  z: 0.8500000238418579
---
header:
  stamp:
    sec: 1780065275
    nanosec: 69270080
  frame_id: base_link
point:
  x: 1.25
  y: 0.949999988079071
  z: 0.8500000238418579
---

Ln 62, Col 31
```

Figura 3.12 Exemplu de valori publicate pe topicul `/utlidar/range_info`

Aceste informații au fost suficiente pentru o strategie reactivă de evitare a obstacolelor. Pentru validare, am realizat mai întâi un program separat bazat doar pe LiDAR, fără detecția YOLO. Scopul a fost să verific reacția robotului atunci când întâlnește obstacole.

Programul citea distanțele din față și din lateral, le compara cu pragurile de siguranță și genera comenzi simple de evitare. Dacă distanța frontală scădea sub pragul stabilit, robotul nu mai mergea înainte și se rotea către partea în care exista mai mult spațiu liber. Un exemplu de comandă generată pe baza datelor LiDAR este prezentat în figura 3.13.

```
^C(.venv) unitree@ubuntu:~/go2_vision$ python3 lidar_range_decision.py
FRONT_BLOCKED -> ROTATE_RIGHT
[INFO] [1780062581.676668573] [lidar_range_decision]: front=0.75 m | left=1.00 m | right=0.95 m |
FRONT_BLOCKED -> ROTATE_LEFT
[INFO] [1780062581.743773463] [lidar_range_decision]: front=0.75 m | left=1.00 m | right=0.95 m |
FRONT_BLOCKED -> ROTATE_LEFT
```

Figura 3.13 Exemplu de comandă de evitare generată pe baza datelor LiDAR

În timpul testelor am observat și situații de colț, în care obstacolele erau prezente atât în față, cât și lateral. În aceste cazuri, simpla rotire la stânga sau la dreapta nu era întotdeauna suficientă, deoarece robotul putea rămâne blocat sau putea oscila între comenzi.

Pentru astfel de situații am introdus un comportament de recuperare. Robotul executa mai întâi o mișcare înapoi, apoi se rotește către zona mai liberă. Această soluție m-a ajutat să evit blocajele și oscilațiile între comenzi.

Prin această etapă am validat separat folosirea LiDAR-ului pentru evitarea obstacolelor, înainte de integrarea completă cu detecția vizuală.

3.3.6 Utilizarea plăcii video pentru procesarea YOLO

După ce am stabilizat arhitectura sistemului, am analizat timpul necesar pentru procesarea unui cadru video. În primele teste, modelul YOLO rula pe procesor. Sistemul funcționa, dar timpul de procesare era destul de mare pentru o aplicație robotică în timp real.

În această configurație, procesarea unui cadru dura aproximativ 0.15–0.20 s. Această întârziere influența reacția robotului, deoarece fiecare comandă era generată după analizarea imaginii. Dacă procesarea dura prea mult, comenzile erau actualizate mai rar, iar robotul reacționa mai lent la schimbările din imagine.

Pentru a reduce această întârziere, am trecut rularea modelului de detecție pe placa video a unității de calcul integrate. După activarea procesării pe GPU, timpul de inferență a scăzut la aproximativ 0.04–0.05 s pentru un cadru.

Această modificare a îmbunătățit comportamentul sistemului. Comenzile au putut fi actualizate mai des, iar robotul a reacționat mai rapid la poziția obiectului detectat.

3.4 Funcționarea sistemului final

După parcurgerea etapelor de implementare, am ajuns la o variantă în care detecția vizuală, datele LiDAR și transmiterea comenzilor funcționează în aceeași buclă. La fiecare ciclu, aplicația citește un cadru video de la cameră, rulează modelul YOLO, citește distanțele furnizate de LiDAR și stabilește comanda care trebuie trimisă robotului.

Funcționarea sistemului este împărțită în mai multe module. Acestea sunt: achiziția imaginilor, detecția obiectului, interpretarea poziției acestuia, verificarea siguranței prin LiDAR, comportamentul de căutare și transmiterea comenzilor. Modulele nu funcționează separat, ci sunt legate într-o singură logică de decizie.

Camera și modelul YOLO oferă informații despre obiectul urmărit. LiDAR-ul oferă informații despre obstacolele din apropierea robotului. Dacă obiectul este detectat, poziția lui în imagine este folosită pentru orientarea robotului. Dacă obiectul nu este detectat, sistemul activează comportamentul de căutare. Dacă LiDAR-ul indică un risc de coliziune, comanda de evitare sau oprire are prioritate.

3.4.1 Achiziția imaginilor și detecția YOLO

Pentru detecția obiectului am folosit o cameră video accesată prin OpenCV. Rezoluția folosită este de 640×480 pixeli. Pentru a reduce întârzierile, sistemul elimină cadrele vechi înainte de procesare, astfel încât YOLO să analizeze o imagine cât mai apropiată de momentul curent.

Modelul folosit pentru detecție este YOLOv8n, configurat pentru detectarea unui scaun. Am păstrat doar detecțiile cu un scor de încredere mai mare de 0.4. Dacă apar mai multe detecții valide, sistemul alege detecția cu scorul cel mai mare.

După detecție, sistemul calculează poziția obiectului în imagine. Cadrul este împărțit logic în trei zone: stânga, centru și dreapta. Dacă obiectul este în stânga sau în dreapta, robotul se orientează în direcția respectivă.

3.4.2 Integrarea datelor LiDAR

Pentru evitarea obstacolelor am folosit topicul ROS2 /utlidar/range_info. Acesta furnizează distanțe relevante pentru zona din fața robotului și pentru zonele laterale.

Datele primite de la LiDAR sunt verificate înainte de a fi folosite. Valorile invalide sunt eliminate, iar situațiile în care senzorul nu detectează obstacole sunt tratate ca spațiu liber.

Sistemul folosește praguri de siguranță pentru a decide dacă robotul poate continua deplasarea. Dacă un obstacol este prea aproape în față, comanda de avansare este blocată. În funcție de distanțele laterale, robotul poate executa o manevră de evitare sau poate intra într-o stare de recuperare.

3.4.3 Logica de decizie și prioritatea comenzilor

Comanda finală este stabilită prin combinarea informațiilor de la YOLO și LiDAR. Modelul de detecție indică poziția obiectului urmărit, iar LiDAR-ul verifică dacă deplasarea este sigură.

Sistemul folosește trei stări principale: starea normală, starea de rotire și starea de recuperare. În starea normală, robotul urmărește obiectul sau îl caută. În starea de rotire, robotul încearcă să evite un obstacol. În starea de recuperare, robotul execută o mișcare de ieșire dintr-o situație de blocaj.

Ordinea comenzilor este importantă. Dacă lipsesc datele de la cameră sau de la LiDAR, robotul se oprește. Dacă există risc de coliziune, evitarea obstacolelor are prioritate. Dacă nu există pericol, robotul urmărește obiectul detectat sau activează comportamentul de căutare.

3.4.4 Transmiterea comenzilor și verificarea vizuală

Comanda finală este publicată pe topicul ROS2 `/go2_command`. De acolo, comanda este preluată de programul `go2_sport_client`, care o transformă într-o mișcare a robotului prin SDK-ul Unitree.

Pentru verificarea funcționării sistemului, aplicația generează o imagine de depanare. Aceasta afișează obiectul detectat, poziția acestuia în imagine, valorile LiDAR și comanda transmisă robotului.

Această imagine a fost utilă în timpul testelor, deoarece mi-a permis să verific rapid dacă decizia sistemului corespunde cu situația reală. De exemplu, am putut observa dacă robotul se rotește din cauza poziției obiectului sau dacă se oprește din cauza unui obstacol detectat de LiDAR.

3.5 Extensie experimentală pentru recunoașterea facială

Pe lângă sistemul principal de detecție a obiectului, am testat și o extensie experimentală pentru recunoașterea unei persoane. Scopul acestei etape a fost să verific dacă sistemul poate fi adaptat astfel încât robotul să caute și să se orienteze către un utilizator cunoscut.

Pentru această testare am folosit biblioteca Python DeepFace, destinată recunoașterii faciale [19]. Sistemul detectează fața din imagine, o compară cu imagini de referință ale persoanei cunoscute și returnează numele persoanei recunoscute împreună cu un scor de similaritate.

În testele preliminare, rezultatul recunoașterii a fost folosit pentru aceeași logică de orientare ca în cazul obiectului detectat. Dacă persoana recunoscută apărea în partea stângă sau dreaptă a imaginii, robotul se orienta către aceasta. Dacă persoana era aproximativ centrată, robotul putea avansa controlat către ea, păstrând verificarea obstacolelor prin LiDAR.

Această extensie nu reprezintă partea principală a lucrării, ci o direcție de dezvoltare aflată într-o fază inițială. Pentru o funcționare robustă sunt necesare teste suplimentare, mai multe imagini de referință și verificarea comportamentului în condiții diferite de iluminare, distanță și poziție a feței.

4 Rezultate experimentale

În acest capitol prezint rezultatele obținute în urma testării sistemului implementat pe robotul patruped Unitree Go2. Am testat principalele componente ale sistemului: detecția obiectului cu YOLO, generarea comenzilor de deplasare, evitarea obstacolelor cu ajutorul LiDAR-ului, comportamentul de căutare și transmiterea comenzilor prin ROS2 către robot.

4.1 Obiectivele testării

Testele au fost organizate în jurul principalelor funcții ale sistemului. Am urmărit verificarea detecției obiectului de interes, interpretarea poziției acestuia în imagine și generarea comenzilor corespunzătoare de orientare, deplasare sau oprire.

De asemenea, am verificat transmiterea comenzilor prin ROS2 către modulul C++ de comandă, folosirea datelor LiDAR pentru evitarea obstacolelor și comportamentul de căutare atunci când obiectul nu este detectat. Un alt obiectiv al testării a fost compararea timpului de procesare al modelului YOLO în varianta cu rulare pe CPU și în varianta cu rulare pe GPU.

4.2 Configurația de test

Am realizat testele pe robotul patruped Unitree Go2, folosind camera video conectată la robot și senzorul LiDAR integrat. Procesarea imaginilor, citirea datelor LiDAR și generarea comenzilor au fost realizate local, pe unitatea de calcul a robotului. Laptopul a fost folosit pentru conectare, monitorizare și lansarea programelor de test.

Pentru detecția obiectului am utilizat modelul YOLO, configurat pentru identificarea unui scaun. Am folosit un prag minim de încredere de 0.4, iar cadrele video au fost preluate la rezoluția de 640×480 pixeli.

Datele LiDAR au fost preluate din topicul ROS2 `/utlidar/range_info`, iar comenzile generate de modulul de decizie au fost transmise prin topicul ROS2 `/go2_command`. Aceste comenzi au fost preluate de programul `go2_sport_client` și transformate în comenzi de mișcare prin SDK-ul Unitree.

4.3 Rezultate privind detecția obiectului

În cadrul testelor am analizat mai multe situații reprezentative, în care am poziționat obiectul urmărit în zone diferite ale imaginii: în partea stângă, în partea dreaptă și în zona centrală. Scopul acestor teste a fost să verific dacă sistemul detectează corect obiectul și dacă poziția acestuia în cadru este interpretată corect de modulul de decizie.

În primul test, am poziționat scaunul în partea stângă față de robot, după cum se observă în figura 4.1. În figura 4.2 se observă rezultatul detecției YOLO. Scaunul este încadrat cu o casetă de detecție, iar centrul obiectului se află în partea stângă față de axa centrală a imaginii. Pe baza acestei poziții, sistemul a generat comanda de rotire spre stânga, pentru a orienta robotul către obiect. Comanda generată este confirmată și în mesaje afișate în terminal, prezentate în figura 4.3.



Figura 4.1 Poziționarea inițială a scaunului în partea stângă față de robot

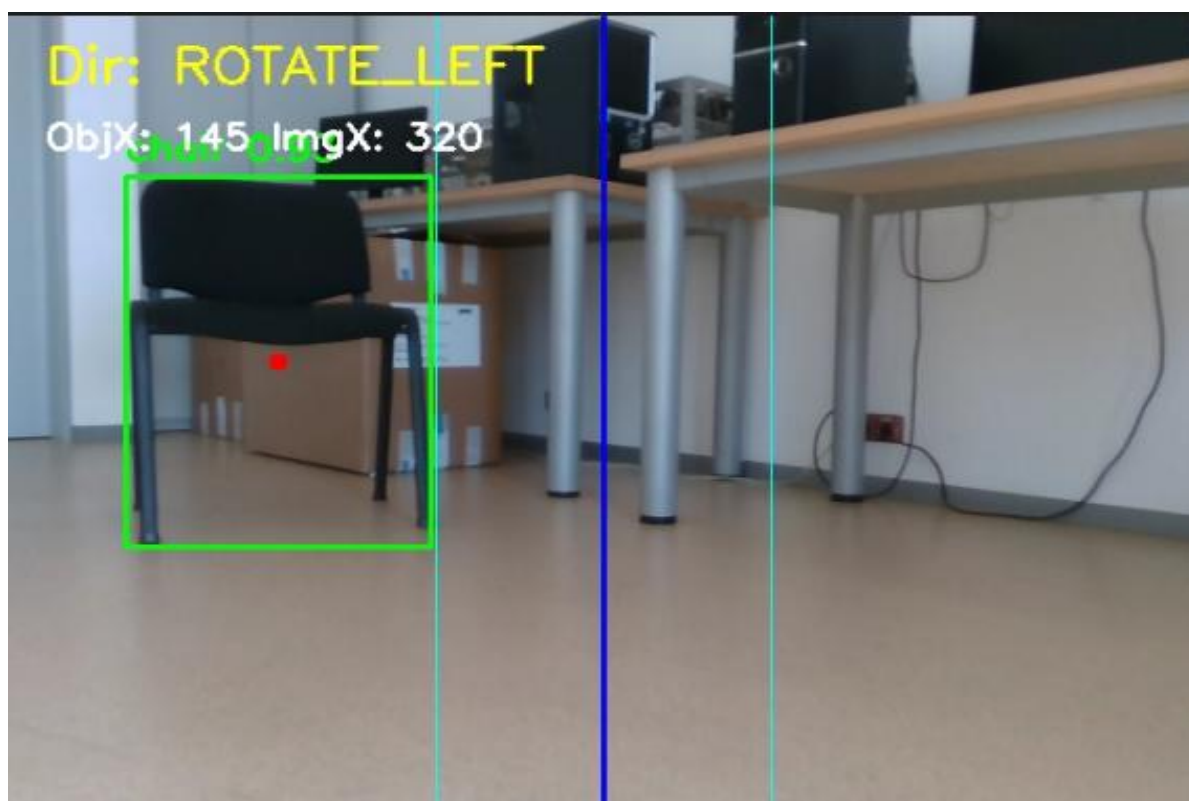


Figura 4.2 Detectarea scaunului în partea stângă a imaginii și generarea comenzii de rotire spre stânga

```
Frame: 1729 | Obj: 68 | Img center: 320 | conf: 0.83  
direction: ROTATE LEFT  
YOLO time: 0.032593727111816406  
Command: ROTATE LEFT  
Frame: 1730 | Obj: 80 | Img center: 320 | conf: 0.88  
direction: ROTATE LEFT
```

Figura 4.3 Mesaje afișate în terminal pentru comanda de rotire spre stânga

După executarea rotației, scaunul ajunge mai aproape de zona centrală a imaginii, după cum se observă în figura 4.4. În momentul în care obiectul este centrat, sistemul nu mai generează o comandă de rotire, ci transmite comanda de oprire.

În figura 4.5 se observă că scaunul este poziționat în zona centrală a cadrului. Mesajele afișate în terminal, prezentate în figura 4.6, confirmă faptul că sistemul a interpretat obiectul ca fiind centrat și a transmis comanda de oprire.



Figura 4.4 Poziționarea robotului după rotirea către scaun

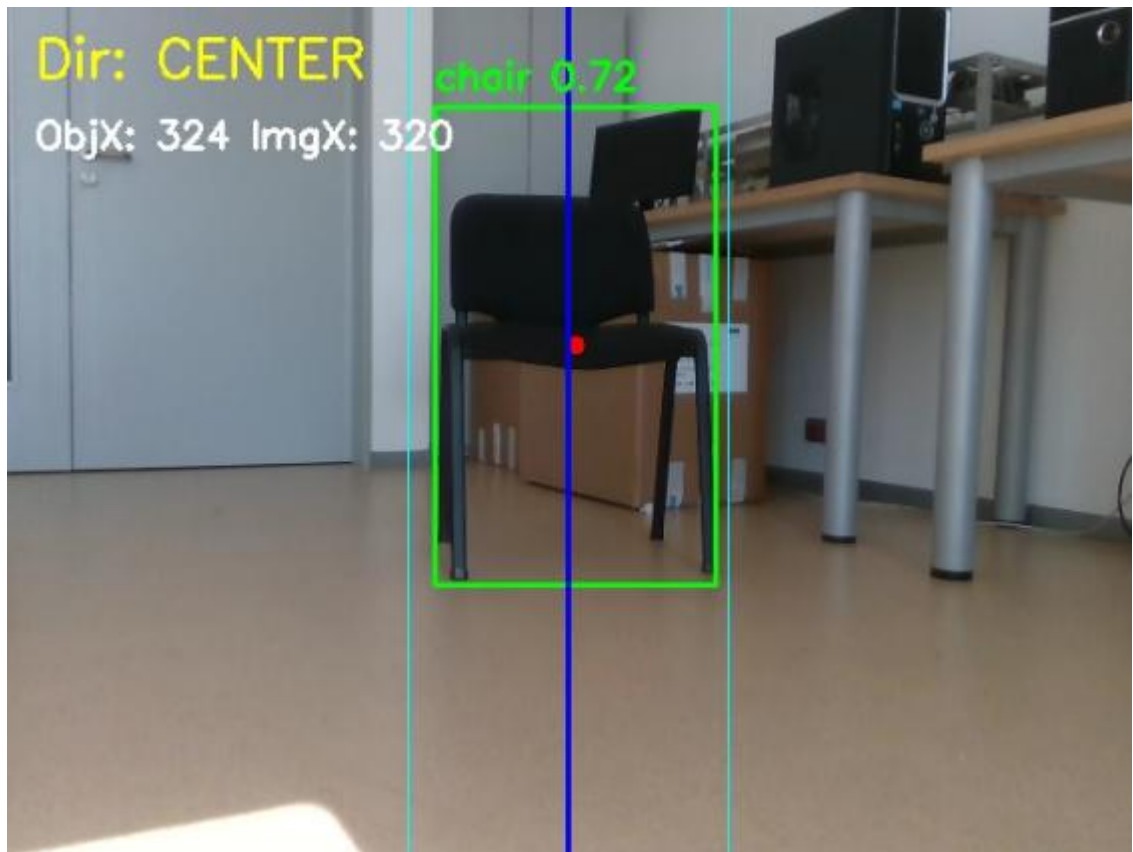


Figura 4.5 Detectarea scaunului în zona centrală a imaginii

```

Command: STOP
Frame: 152 | Obj: 325 | Img center: 320 | conf: 0.69
direction: CENTER
YOLO time: 0.0426325798034668
Command: STOP
Frame: 153 | Obj: 324 | Img center: 320 | conf: 0.80
  
```

Figura 4.6 Mesaje afișate în terminal pentru comanda de oprire

Pentru cazul în care scaunul a fost poziționat în partea dreaptă a imaginii cum se observă în figura 4.7, comportamentul sistemului a fost similar. Sistemul a detectat obiectul, a calculat poziția acestuia față de centrul cadrului și a generat comanda de rotire spre dreapta. Prin acest test am verificat că modulul de decizie funcționează corect și pentru situația opusă, orientând robotul în direcția în care se află obiectul urmărit.

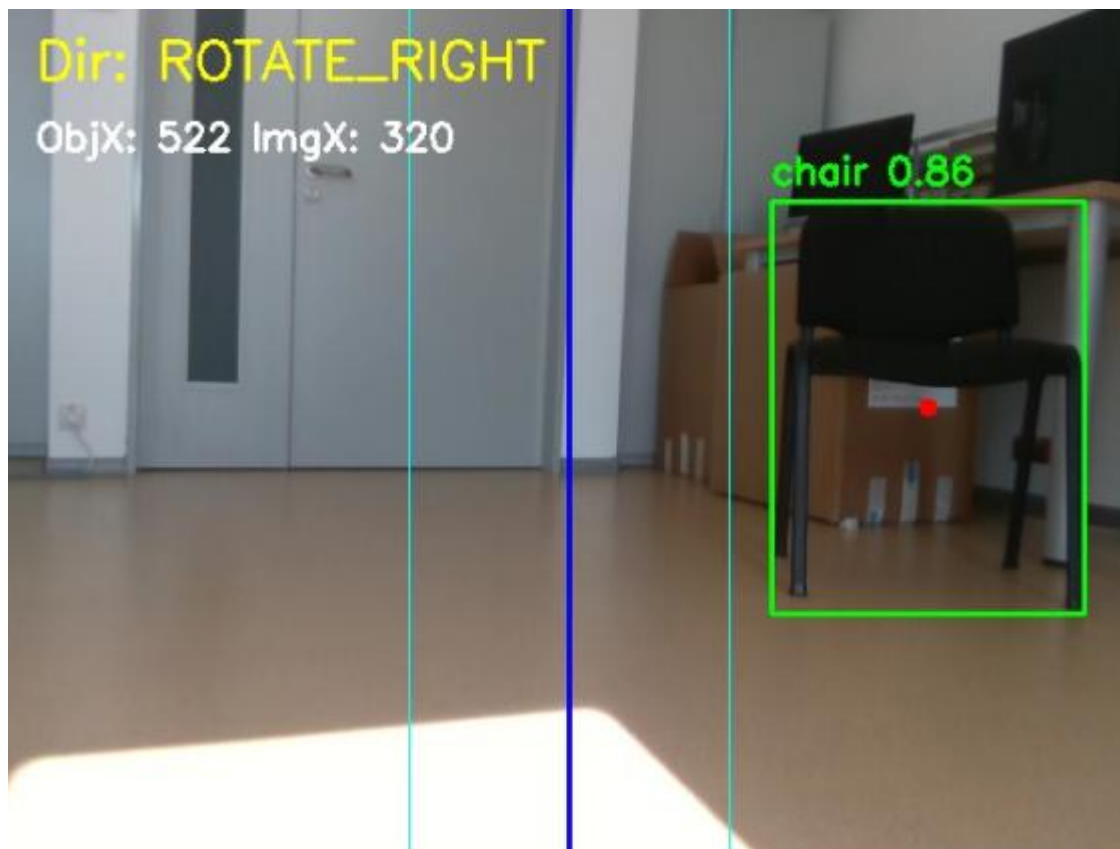


Figura 4.7 Detectarea scaunului în partea dreaptă a imaginii și generarea comenzii de rotire spre dreapta

Prin acest test am verificat că sistemul poate detecta obiectul urmărit, poate interpreta poziția acestuia în imagine și poate genera comenzi corecte de orientare sau oprire.

4.4 Rezultate privind evitarea obstacolelor cu LiDAR

În această etapă am testat modulul de evitare a obstacolelor folosind datele furnizate de senzorul LiDAR. Scopul testului a fost să verific dacă robotul poate interpreta corect distanțele din jurul său și dacă poate genera comenzi de deplasare în funcție de obstacolele detectate.

În primul scenariu, robotul a avut spațiu liber în față, după cum se observă în figura 4.8. Datele LiDAR vizualizate în RViz2, prezentate în figura 4.9, au confirmat faptul că zona frontală era liberă. În această situație, sistemul a transmis comanda de deplasare înainte, lucru confirmat de mesajele afișate în terminal în figura 4.10.

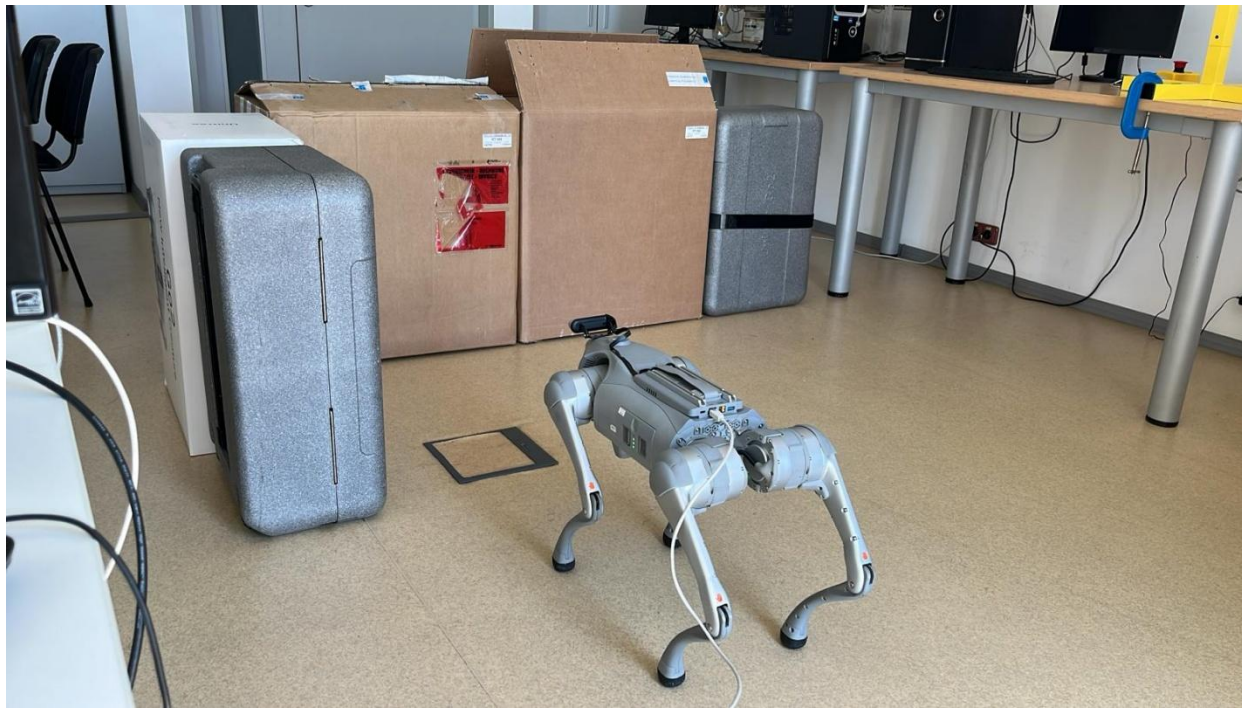


Figura 4.8 Scenariul inițial de test, cu spațiu liber în fața robotului

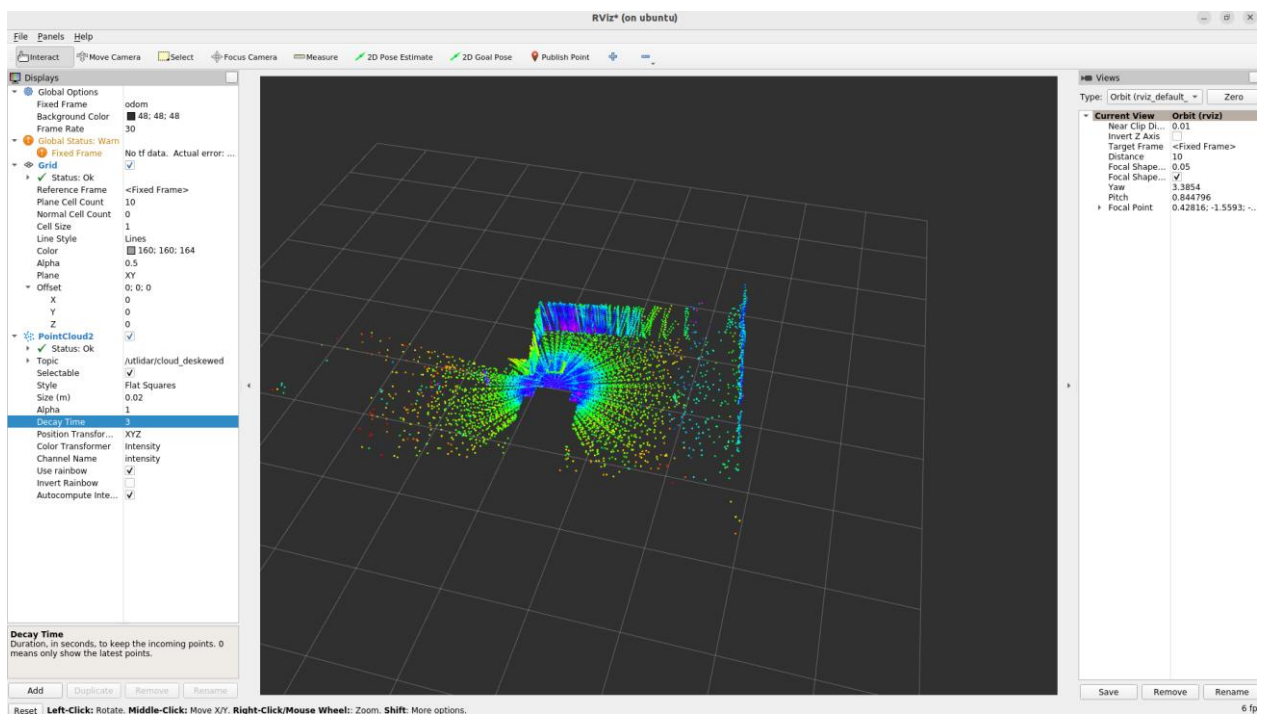


Figura 4.9 Vizualizarea datelor LiDAR în RViz2 pentru scenariul cu direcția frontală liberă

```
[INFO] [1781871871.985834334] [lidar_avoid_simple]: mode=NORMAL | front=1.35 | left=0.85 | right=1.90 | blocked=0  
command=FORWARD  
[INFO] [1781871872.435795502] [lidar_avoid_simple]: mode=NORMAL | front=1.35 | left=0.85 | right=1.60 | blocked=0  
command=FORWARD
```

Figura 4.10 Mesaje afișate în terminal pentru comanda de deplasare înainte

După deplasarea înainte, robotul s-a apropiat de obstacolul aflat în partea stângă, după cum se observă în figura 4.11. În această situație, datele LiDAR au indicat o distanță laterală stânga mai mică decât pragul stabilit. Vizualizarea din RViz2, prezentată în figura 4.12, arată obstacolele din apropierea robotului. Ca urmare, sistemul a generat comanda de deplasare spre dreapta, pentru a corecta poziția robotului și pentru a evita apropierea excesivă de obstacol. Această decizie este confirmată de mesajele afișate în terminal în figura 4.13.



Figura 4.11 Aproximarea robotului de obstacolul lateral stânga

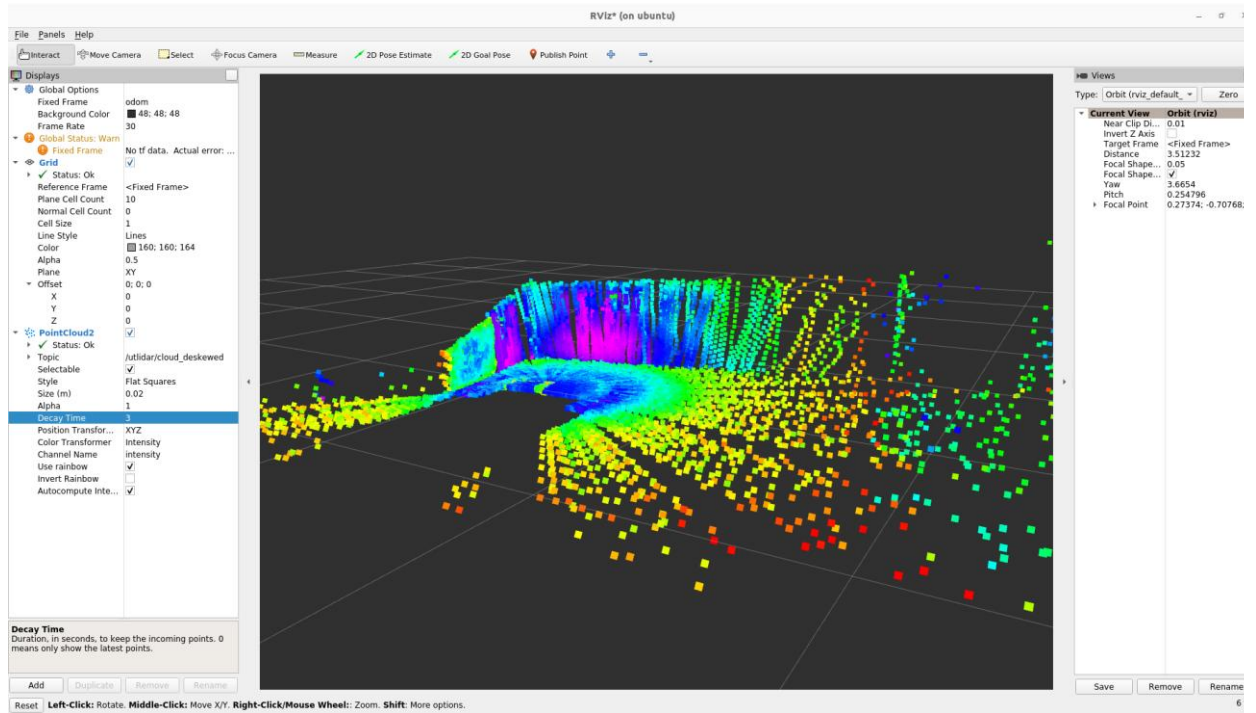


Figura 4.12 Vizualizarea datelor LiDAR în RViz2 pentru apropierea de obstacolul lateral

```
[INFO] [1781871876.935845742] [lidar_avoid_simple]: mode=NORMAL | front=1.20 | left=0.30 | right=1.80 | blocked=0
command=RIGHT
[INFO] [1781871877.385824967] [lidar_avoid_simple]: mode=NORMAL | front=1.20 | left=0.30 | right=1.55 | blocked=0
command=RIGHT
```

Figura 4.13 Mesaje afișate în terminal pentru comanda de deplasare spre dreapta

Într-un alt scenariu, robotul s-a apropiat de zona frontală blocată, după cum se observă în figura 4.14. Datele LiDAR vizualizate în RViz2, prezentate în figura 4.15, indică faptul că în fața robotului există un obstacol apropiat. Deoarece distanța frontală a scăzut sub pragul de siguranță, sistemul a blocat deplasarea înaintea și a generat o comandă de rotire către direcția în care exista mai mult spațiu disponibil. Această decizie este prezentată în mesajele din terminal în figura 4.16.



Figura 4.14 Scenariu în care robotul se apropie de un obstacol frontal

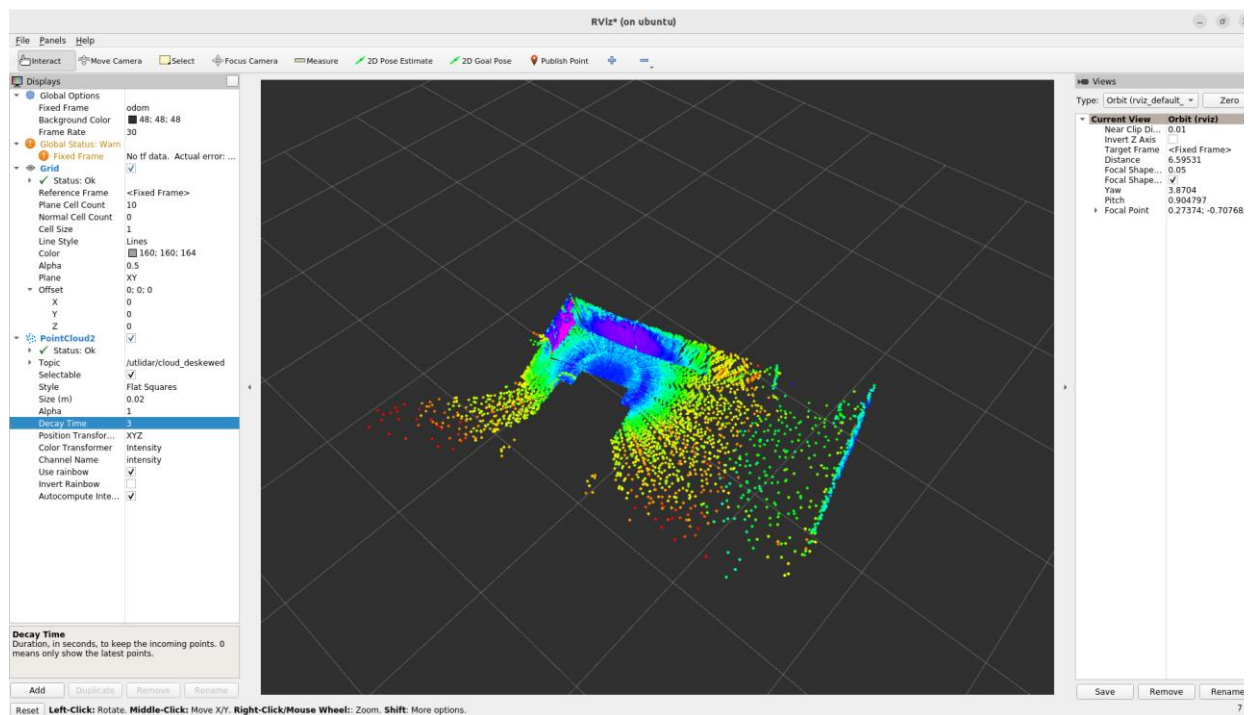


Figura 4.15 Vizualizarea datelor LiDAR în RViz2 pentru obstacolul frontal

```

Command=ROTATE RIGHT
INFO [1781871891.785648948] [lidar_avoid_simple]: mode=RECOVERY | front=1.05 | left=0.40 | right=2.45 | blocked=0
| command=ROTATE RIGHT
INFO [1781871892.236005763] [lidar_avoid_simple]: mode=RECOVERY | front=1.10 | left=0.35 | right=2.75 | blocked=0
| command=ROTATE RIGHT

```

Figura 4.16 Mesaje afișate în terminal pentru comanda de rotire spre dreapta

După executarea rotației, zona frontală a devenit din nou liberă, după cum se observă în figura 4.17. Vizualizarea LiDAR din RViz2, prezentată în figura 4.18, confirmă faptul că robotul a reușit să își modifice orientarea și să elibereze direcția frontală. În această situație, sistemul a putut reveni la deplasarea înainte.

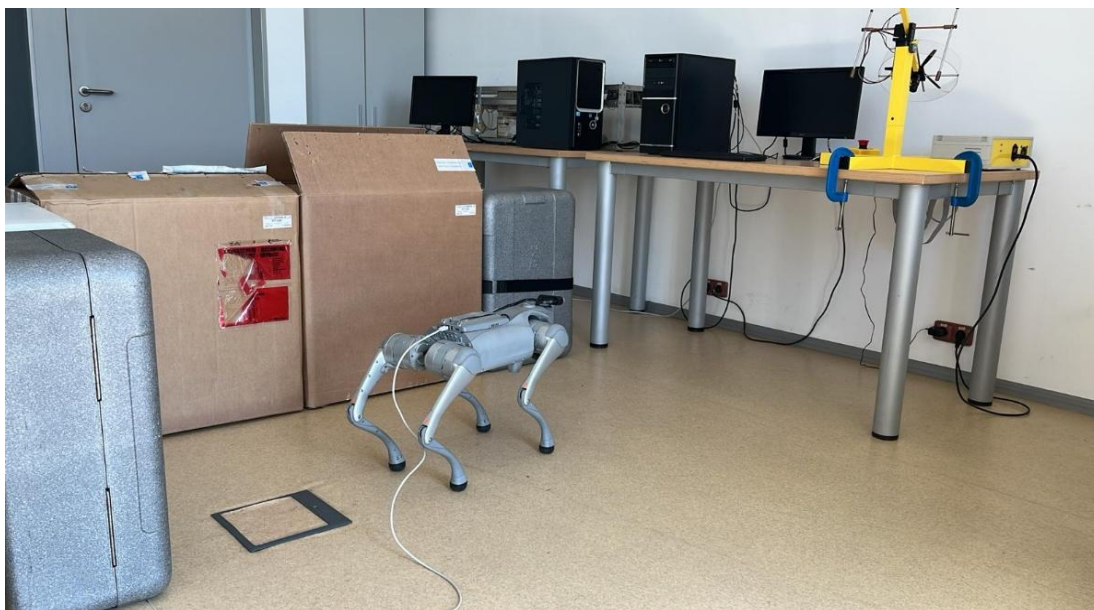


Figura 4.17 Poziția robotului după rotirea de evitare

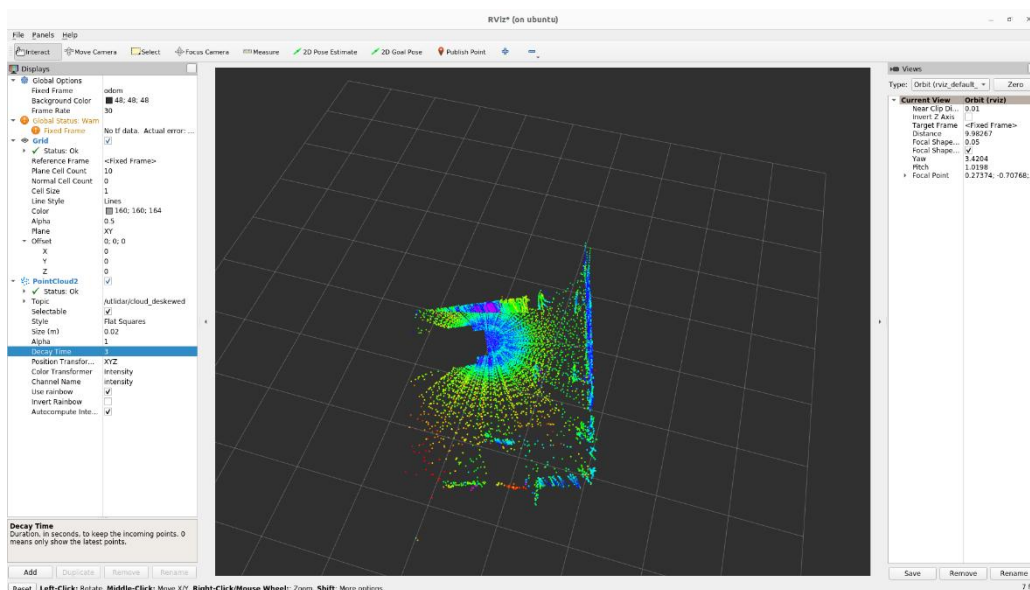


Figura 4.18 Vizualizarea datelor LiDAR după eliberarea direcției frontale

Prin aceste teste am verificat că robotul poate folosi datele LiDAR pentru evitarea obstacolelor frontale și laterale. Sistemul nu se bazează doar pe comanda de deplasare înainte, ci adaptează mișcarea în funcție de distanțele măsurate în jurul robotului.

4.5 Testarea sistemului integrat

În testele finale am urmărit funcționarea simultană a detecției YOLO, a evitării obstacolelor cu LiDAR și a comportamentului de căutare. Scopul acestor teste a fost să verific dacă sistemul poate combina informațiile vizuale cu datele de distanță și poate genera o comandă finală corectă pentru robot.

În cazul prezentat, la începutul testului scaunul nu este vizibil în fața robotului. În figura 4.19 se observă că sistemul nu detectează obiectul urmărit, iar interfața indică faptul că obiectul căutat nu a fost găsit. Deoarece nu există o detecție validă, sistemul activează comportamentul de căutare. În această etapă, comanda generată este de deplasare înainte, deoarece datele LiDAR indică faptul că zona frontală este liberă.

Poziția robotului în timpul acestei etape este prezentată în figura 4.20. Robotul se află pe coridor și începe deplasarea controlată pentru a explora zona din față. Datele LiDAR vizualizate în RViz2 sunt prezentate în figura 4.21 și confirmă faptul că robotul are spațiu disponibil pentru deplasare.



Figura 4.19 Obiectul nu este detectat, iar sistemul activează căutarea

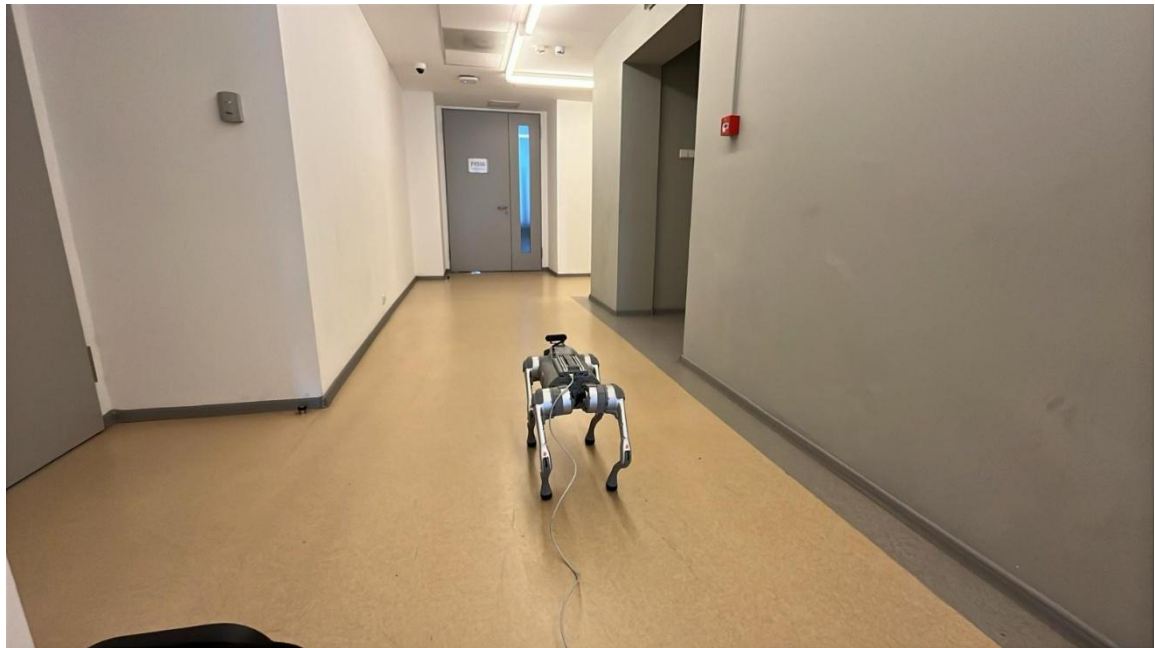


Figura 4.20 Poziția robotului în timpul căutării

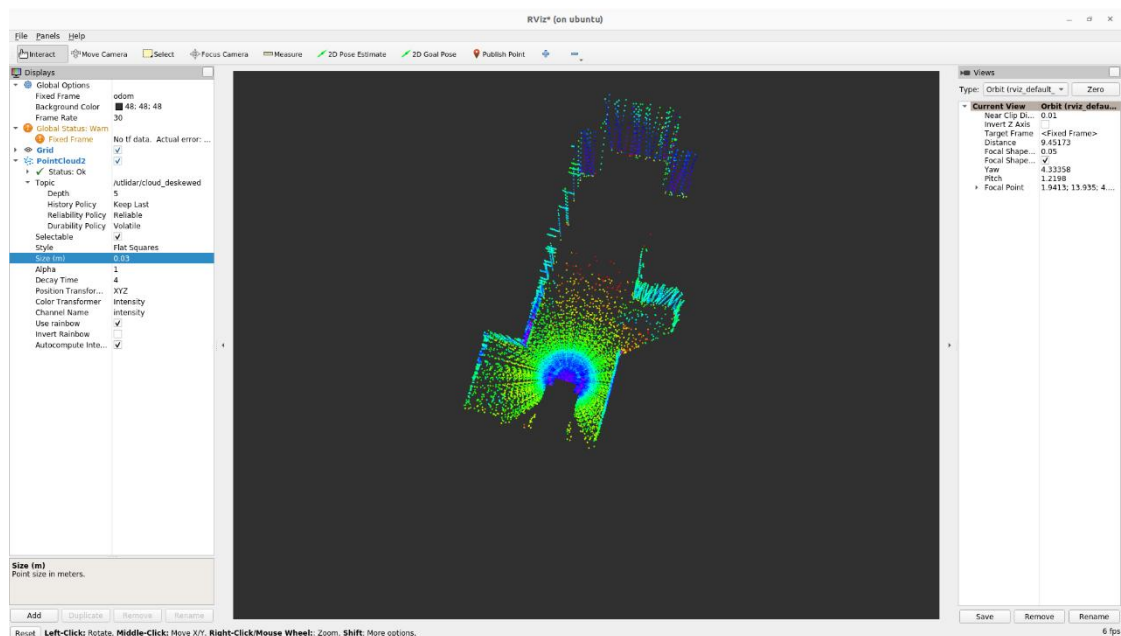


Figura 4.21 Datele LiDAR pentru deplasarea înainte

După deplasarea înainte pentru un număr de cicluri, sistemul continuă comportamentul de căutare prin rotirea robotului. În figura 4.22 se observă că obiectul încă nu este detectat, iar comanda generată este de rotire spre stânga. Această comandă are rolul de a modifica direcția camerei și de a aduce alte zone ale mediului în câmpul vizual. Poziția robotului în această etapă este prezentată în figura 4.23.



Figura 4.22 Rotirea spre stânga în timpul căutării

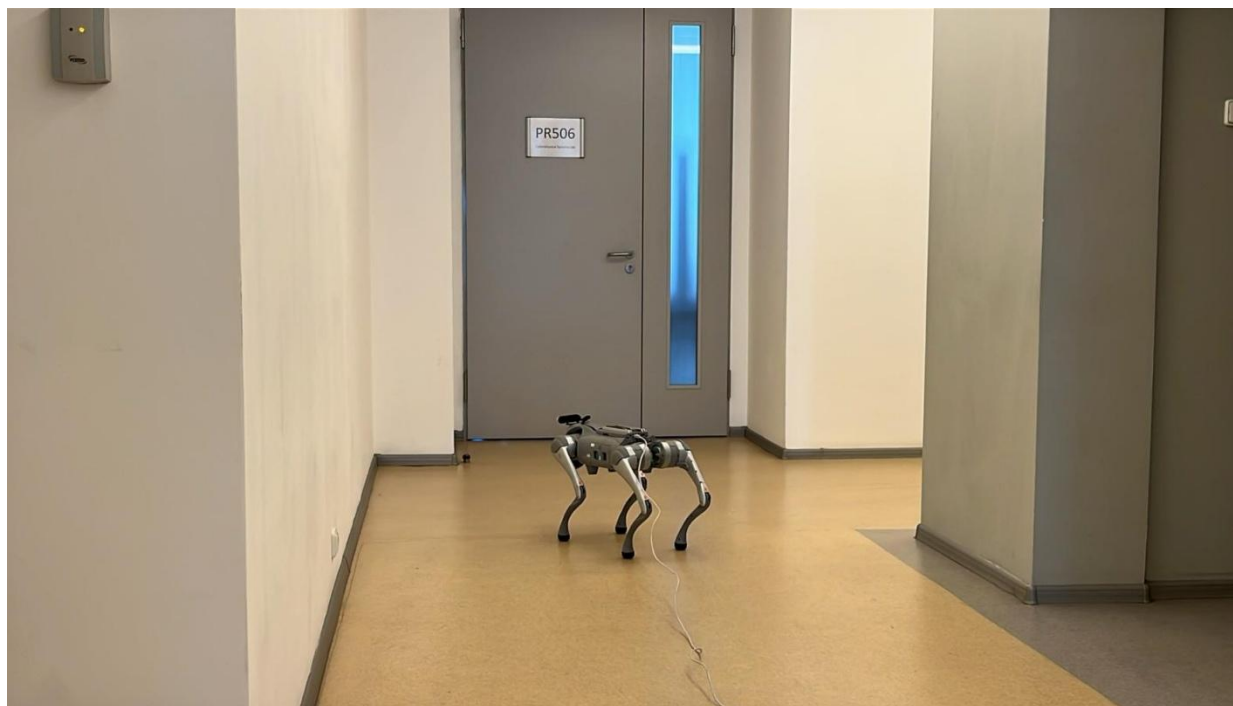


Figura 4.23 Poziția robotului în timpul rotirii

Deoarece scaunul nu este detectat nici după rotirea spre stânga, sistemul continuă secvența de căutare și generează comanda de rotire spre dreapta. Acest lucru este prezentat în figura 4.24, unde se observă în continuare faptul că obiectul urmărit nu a fost identificat, iar comanda curentă este de rotire spre dreapta.



Figura 4.24 Rotirea spre dreapta în timpul căutării

După mai multe cicluri de căutare, scaunul intră în câmpul vizual al camerei. În figura 4.25 se observă că modelul YOLO detectează scaunul cu un scor de încredere ridicat. În același timp, datele LiDAR indică faptul că robotul este prea aproape de peretele din stânga. Din acest motiv, deși obiectul este detectat, comanda finală nu este doar o comandă de urmărire vizuală, ci o comandă de corecție spre dreapta. Vizualizarea LiDAR corespunzătoare este prezentată în figura 4.26.

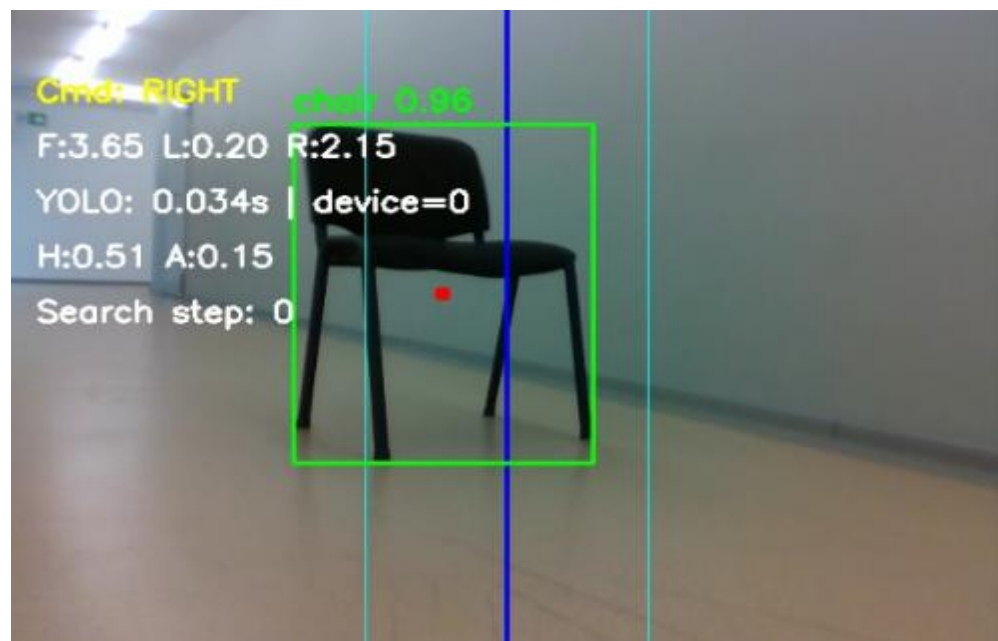


Figura 4.25 Detectarea scaunului și corecția spre dreapta

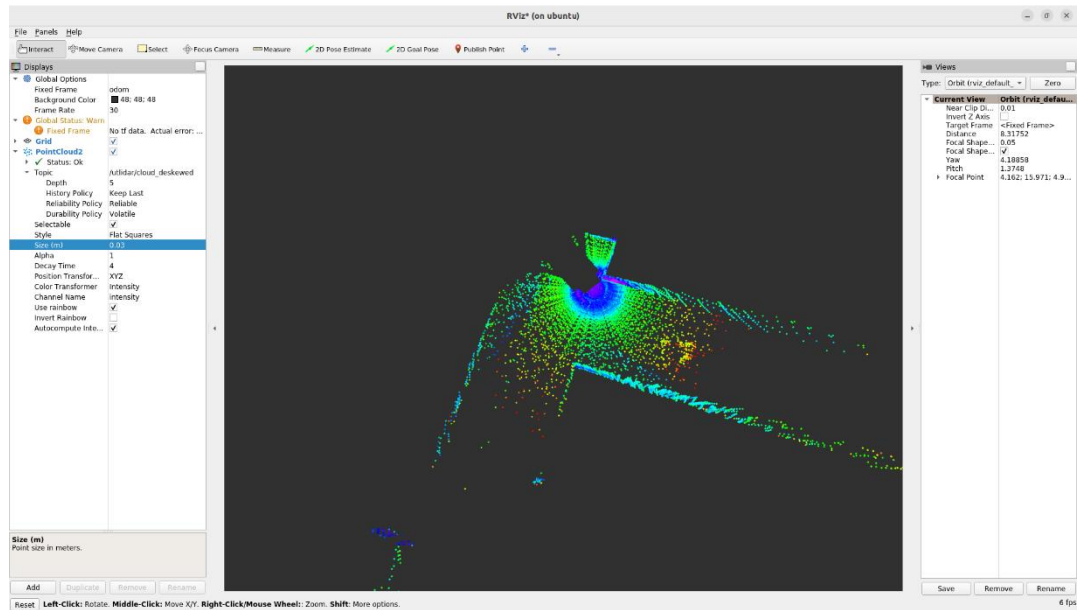


Figura 4.26 Datele LiDAR în apropierea peretelui lateral

După corectarea traiectoriei, robotul ajunge aproape de scaun, după cum se observă în figura 4.27. În această etapă, sistemul a combinat comportamentul de căutare, detecția YOLO și verificarea LiDAR pentru a ghida robotul către obiectul urmărit.

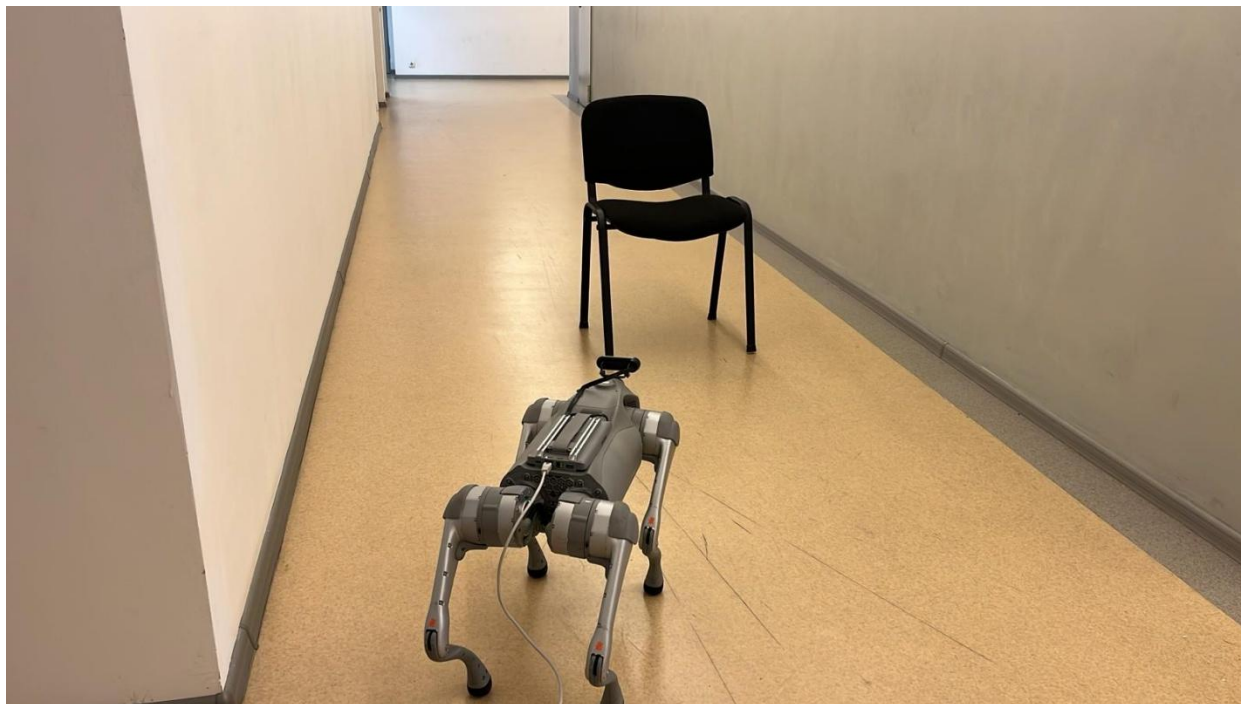


Figura 4.27 Poziția robotului lângă scaun

În figura 4.28 se observă condiția de oprire. Scaunul este detectat cu un scor de încredere ridicat, iar dimensiunea casetei de încadrare indică faptul că obiectul este suficient de aproape. În implementare, robotul se oprește atunci când obiectul ocupă

cel puțin 78% din înălțimea cadrului video sau cel puțin 42% din aria totală a cadrului. În cazul prezentat, valoarea pentru înălțime depășește pragul stabilit, iar sistemul transmite comanda de oprire.

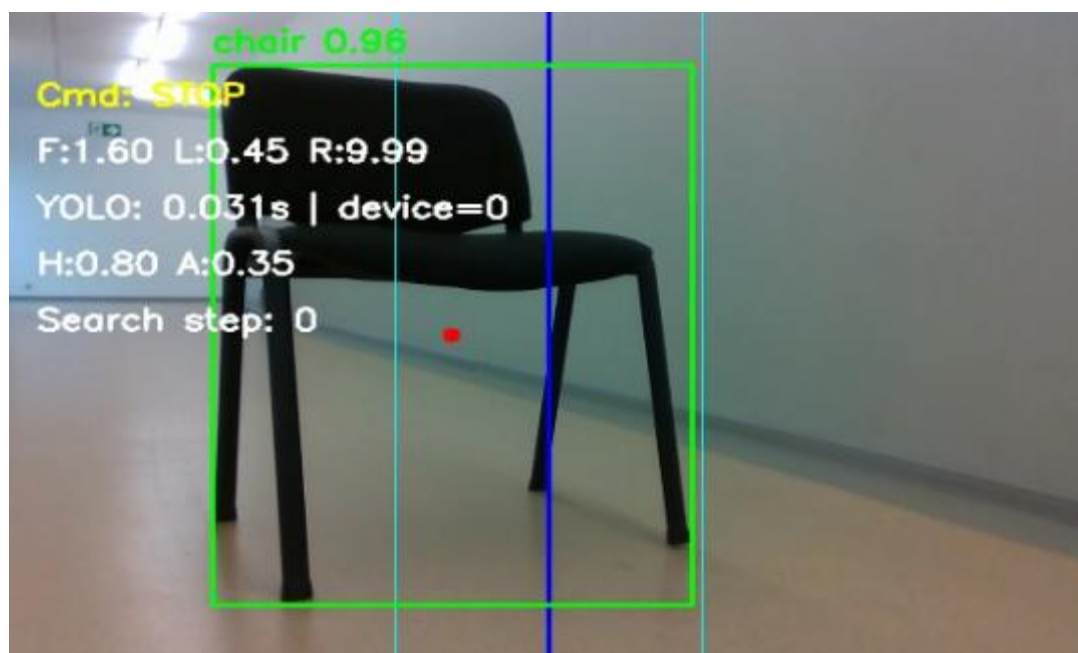


Figura 4.28 Detectarea scaunului și oprirea robotului

4.6 Rezultate privind timpul de procesare

Am comparat timpul de procesare al modelului YOLO în două situații: rulare pe procesor și rulare pe placa video. În varianta inițială, în care inferența era realizată pe CPU, procesarea unui cadru dura aproximativ 0.15–0.20 s. Un exemplu de timp de procesare obținut în această configurație este prezentat în figura 4.29.

După trecerea la procesarea pe GPU, timpul de inferență a scăzut la aproximativ 0.04–0.05 s pentru un cadru, după cum se observă în figura 4.30. Această reducere a timpului de procesare a permis actualizarea mai rapidă a comenzilor și a îmbunătățit reacția robotului la schimbările din imagine.

```
Command: STOP  
Frame: 8 | Obj: 290 | Img center: 320 | conf: 0.84  
direction: CENTER  
YOLO time: 0.21826410293579102
```

Figura 4.29 Exemplu de timp de procesare YOLO în varianta cu rulare pe CPU

Frame 722	CHAIR FOUND	conf=0.92	YOLO time=0.051s	FPS=19.65	device=0
Frame 723	CHAIR FOUND	conf=0.92	YOLO time=0.043s	FPS=23.18	device=0
Frame 724	CHAIR FOUND	conf=0.92	YOLO time=0.045s	FPS=22.43	device=0
Frame 725	CHAIR FOUND	conf=0.92	YOLO time=0.049s	FPS=20.58	device=0

Figura 4.30 Exemplu de timp de procesare YOLO în varianta cu rulare pe GPU

4.7 Test preliminar pentru recunoașterea facială

Pe lângă testele principale cu detecția obiectului, am realizat și un test separat pentru recunoașterea facială. Scopul a fost să verific dacă aceeași arhitectură poate fi adaptată astfel încât robotul să recunoască o persoană cunoscută și să folosească poziția acesteia în imagine pentru orientare sau deplasare.

Pentru această etapă am folosit biblioteca Python DeepFace. Programul detectează fața din imagine și o compară cu imaginile de referință salvate anterior. În cazul unei potriviri, sistemul returnează numele persoanei recunoscute și un scor de similaritate.

În figura 4.31 este prezentat un exemplu în care persoana este recunoscută corect. Pe imagine sunt afișate numele persoanei, scorul obținut și poziția feței față de centrul cadrului. Această informație este importantă deoarece permite folosirea aceleiași logici de orientare ca în cazul obiectului detectat.



Figura 4.31 Recunoașterea persoanei și determinarea poziției acesteia față de centrul imaginii

În figura 4.32 este prezentat un caz în care rezultatul recunoașterii este folosit pentru generarea unei comenzi de mișcare. Dacă fața recunoscută se află în stânga sau în dreapta imaginii, robotul se poate orienta în direcția respectivă. Dacă persoana este aproximativ centrată, robotul poate avansa controlat, cu verificarea obstacolelor prin LiDAR. În exemplul prezentat, persoana este recunoscută ca țintă, iar sistemul transmite comanda de deplasare înainte.

Această parte a fost tratată ca o extensie experimentală, nu ca funcționalitate principală a lucrării. Testul arată că sistemul poate fi extins de la detectarea unui obiect

la recunoașterea unei persoane cunoscute. Pentru o funcționare mai stabilă ar fi necesare mai multe imagini de referință.

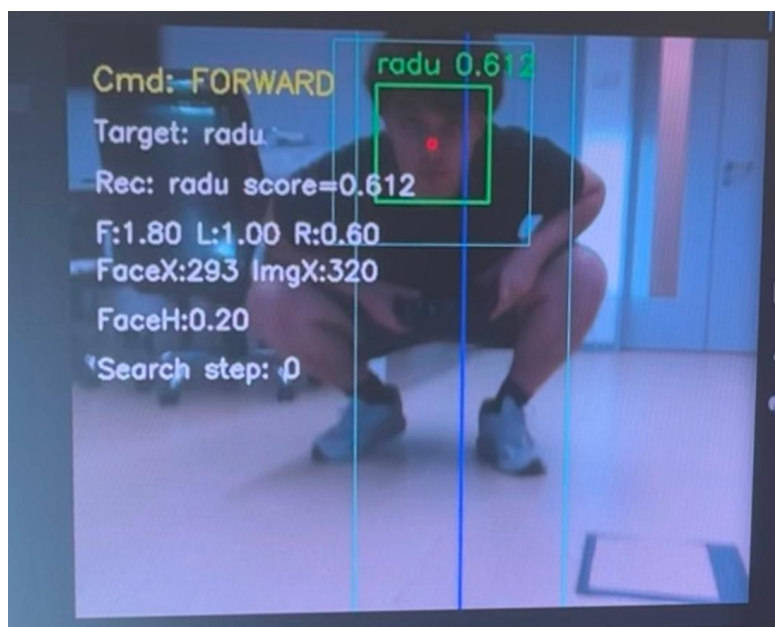


Figura 4.32 Generarea comenzii de deplasare pe baza recunoașterii persoanei

4.8 Limitări observate

În timpul testelor au apărut câteva limitări ale sistemului implementat. Prima este legată de detecția vizuală. Modelul YOLO a funcționat bine atunci când obiectul era clar vizibil în imagine, dar detecția a devenit mai instabilă în situații cu iluminare diferită, unghiuri mai dificile sau atunci când obiectul era parțial acoperit.

O altă limitare este faptul că robotul nu calculează poziția exactă a obiectului în spațiul tridimensional. Sistemul folosește doar poziția obiectului în imagine pentru a decide direcția de mișcare. Din acest motiv, apropierea de obiect se face printr-o logică reactivă, pe baza centrării în imagine și a unor praguri stabilite experimental.

Și evitarea obstacolelor este realizată tot reactiv. Robotul folosește distanțele curente primite de la LiDAR pentru a decide dacă poate avansa, dacă trebuie să se rotească sau dacă trebuie să corecteze lateral. Sistemul nu construiește o hartă a mediului și nu planifică un traseu complet. Această abordare a fost suficientă pentru scenariile testate, dar în spații mai aglomerate poate duce la mișcări mai puțin fluide sau la situații în care robotul trebuie să se re poziționeze.

Comportamentul de căutare este, de asemenea, simplu. Robotul alternează între deplasări scurte și rotații, pentru a aduce obiectul în câmpul vizual al camerei. Această strategie funcționează în zona apropiată, dar nu garantează găsirea obiectului dacă acesta este ascuns, foarte departe sau aflat într-o direcție pe care robotul nu o explorează suficient.

Modulul de recunoaștere facială a fost tratat ca o extensie experimentală. În testele preliminare, acesta a putut recunoaște o persoană cunoscută, dar rezultatele pot varia în funcție de lumină, distanță, poziția feței și calitatea imaginilor de referință. Din acest motiv, această funcționalitate nu a fost inclusă ca parte principală a sistemului final.

5 Concluzii și dezvoltări viitoare

În această lucrare am realizat un sistem software pentru robotul patruped Unitree Go2, prin care acesta poate detecta un obiect de interes, îl poate căuta atunci când nu se află în câmpul vizual și se poate apropia controlat de el. Sistemul folosește detecția vizuală cu YOLOv8n, datele LiDAR pentru verificarea obstacolelor și un modul C++ bazat pe SDK-ul Unitree, care transformă comenzile primite prin ROS2 în mișcări ale robotului.

Contribuția principală a lucrării constă în integrarea acestor componente într-un sistem funcțional, testat pe robot. Modulul Python preia imaginea de la cameră, rulează detecția YOLO, interpretează poziția obiectului și verifică datele LiDAR. Comanda rezultată este publicată pe topicul ROS2 /go2_command, iar modulul go2_sport_client o transformă într-o comandă executabilă de robot.

Implementarea a fost dezvoltată treptat. Prima variantă a folosit procesarea externă pe laptop, apoi detecția a fost mutată pe unitatea de calcul a robotului. Ulterior, transmiterea comenzilor prin *subprocess* a fost înlocuită cu un publisher ROS2, iar sistemul a fost completat cu date LiDAR pentru evitarea obstacolelor.

Testele realizate au arătat că robotul poate detecta obiectul urmărit, poate interpreta poziția acestuia în imagine și poate genera comenzi de orientare, deplasare sau oprire. De asemenea, folosirea LiDAR-ului a permis blocarea sau modificarea comenzilor atunci când în fața robotului sau în zonele laterale au fost detectate obstacole apropiate. În varianta integrată, sistemul a combinat detecția obiectului, comportamentul de căutare și evitarea obstacolelor într-o singură buclă de funcționare.

Ca dezvoltări viitoare, sistemul poate fi îmbunătățit printr-o estimare mai bună a distanței față de obiect și printr-un comportament de căutare mai robust. O altă direcție este dezvoltarea modului de recunoaștere facială, astfel încât robotul să poată identifica mai stabil o persoană cunoscută. În plus, sistemul ar putea fi extins cu recunoașterea unor gesturi simple, precum oprire, apropiere sau schimbarea direcției, pentru o interacțiune mai naturală cu robotul.

În concluzie, lucrarea arată că robotul Unitree Go2 poate fi folosit într-un sistem autonom de percepție și comandă, bazat pe cameră, YOLO, LiDAR, ROS2 și SDK-ul Unitree.

6 Listă contribuții personale

Sistem autonom de detecție, căutare și evitare a obstacolelor pentru robotul patruped Unitree Go2

Bălăceanu Radu

Nr. crt.	Activitate	Durată [zile]
1	Documentare privind roboții patrupezi, Unitree Go2, YOLO, LiDAR, ROS2 și SDK	10
2	Configurarea mediului software și testarea conexiunii cu robotul	7
3	Implementarea detecției YOLO și testarea procesării imaginilor	6
4	Implementarea transducerii comenzilor prin <i>subprocess</i> și apoi prin publisher ROS2	6
5	Adaptarea modulului C++ <i>go2_sport_client</i> pentru comenzi primite prin ROS2	2
6	Integrarea datelor LiDAR și implementarea evitării obstacolelor	16
7	Implementarea comportamentului de căutare și a logicii finale de decizie	9
8	Optimizarea timpului de procesare prin folosirea GPU	1
9	Testarea sistemului integrat pe robot și analiza rezultatelor	8
10	Test preliminar pentru recunoașterea facială	2
11	Redactarea lucrării și organizarea rezultatelor experimentale	14

7 Bibliografie

- [1] Szeliski R. 2022. *Computer Vision: Algorithms and Applications*. Springer Cham, doi: 10.1007/978-3-030-34372-9.
- [2] Corke P. 2023. *Robotics, Vision and Control: Fundamental Algorithms in Python*. Springer Cham, doi: 10.1007/978-3-031-06469-2.
- [3] Redmon J., Divvala S., Girshick R., Farhadi A. 2016. *You Only Look Once: Unified, Real-Time Object Detection*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, p. 779–788, doi: 10.48550/arXiv.1506.02640.
- [4] Girshick R., Donahue J., Darrell T., Malik J. 2014. *Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, p. 580–587.
- [5] Girshick R. 2015. *Fast R-CNN*. Proceedings of the IEEE International Conference on Computer Vision, p. 1440–1448.
- [6] Ren S., He K., Girshick R., Sun J. 2015. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. Advances in Neural Information Processing Systems, vol. 28, p. 91–99.
- [7] Liu W., Anguelov D., Erhan D., Szegedy C., Reed S., Fu C.-Y., Berg A.C. 2016. *SSD: Single Shot MultiBox Detector*. European Conference on Computer Vision, Springer, p. 21–37.
- [8] Bradski G. 2000. *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
- [9] Ultralytics. 2026. *YOLOv8 Documentation*. Documentație oficială online, <https://docs.ultralytics.com/models/yolov8/>, consultat în 2026.
- [10] OpenCV team. 2026. *OpenCV Documentation*. Documentație oficială online, <https://docs.opencv.org/>, consultat în 2026.
- [11] Open Robotics. 2026. *ROS 2 Documentation: Nodes, Topics and Publisher/Subscriber Communication*. Documentație oficială online, <https://docs.ros.org/>, consultat în 2026.
- [12] Object Management Group. 2015. *Data Distribution Service Version 1.4 Specification*. OMG, <https://www.omg.org/spec/DDS/1.4/>, consultat în 2026.
- [13] Unitree Robotics. 2026. *Unitree ROS2 Documentation and SDK2 Repository*. Documentație oficială online, https://github.com/unitreerobotics/unitree_ros2, consultat în 2026.
- [14] Unitree Robotics. 2026. *Unitree SDK2 Repository*. Documentație oficială online, https://github.com/unitreerobotics/unitree_sdk2, consultat în 2026.
- [15] Boston Dynamics. 2026. *Spot SDK Documentation*. Documentație oficială online, <https://dev.bostondynamics.com/>, consultat în 2026.

[16] ANYbotics. 2026. *ANYmal Autonomous Robotic Inspection Solution*. Documentație oficială online, <https://www.anybotics.com/robotics/anymal/>, consultat în 2026.

[17] ANYbotics. 2024. *Why is an IP67 rating critical for inspection robots?* Articol tehnic online, <https://www.anybotics.com/news/effective-inspection-robots-demand-robust-resilient-design/>, consultat în 2026.

[18] Boston Dynamics. 2026. *Spot Tensorflow Object Detection*. Documentație oficială online, disponibil la: https://dev.bostondynamics.com/python/examples/spot_tensorflow_detector/README.html, consultat în 2026.

[19] Serengil S.I., Ozpinar A. 2024. *DeepFace*. Bibliotecă Python pentru recunoaștere facială. Disponibil la: <https://github.com/serengil/deepface>. Consultat în 2026.